

10

Building a Real-Time Game

In this chapter, we will build a multi-player, head-to-head game app with real-time communication. In the app, you will be able to connect to a game server and view a list of other players that are also connected. You can then select a player to request a game with them and, provided they accept, play a game of *Sticks & Stones*. We will look at how we can use SignalR to implement a real-time connection with the server.

The following topics will be covered in this chapter:

- How to use SignalR in a .NET MAUI app
- How to use control templates
- How to use XAML triggers to update the interface
- How to use XAML styling in a .NET MAUI app

Let's get started.

Technical requirements

Before you start building the app for this project, you need to build the backend that we detailed in *Chapter 9, Setting Up a Backend for a Game Using Azure Services*. You will also need to have Visual Studio for Mac or PC installed, as well as the .NET MAUI components. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment. The source code for this chapter is available in this book's GitHub repository: <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition/tree/chapters/ten/main>.

Project overview

When building a head-to-head game app, it is really important to have real-time communication because the user expects the other players' moves to arrive more or less immediately. To achieve this, we will use SignalR, which is a library for real-time communication. SignalR will use WebSockets if they are available and, if not, it will have several fallback options it can use instead. In the app, we will use SignalR to send updates on player and game status through the Azure Functions that we built in *Chapter 9*.

The build time for this project is about 180 minutes.

Getting started

We can use either Visual Studio on a PC or Mac to complete this project. To build an iOS app using Visual Studio for PC, you have to have a Mac connected. If you don't have access to a Mac at all, you can choose to just build the Android part of the app.

Let's review from *Chapter 9* what the game is all about.

An overview of the game

Sticks & Stones is a turn-based social game based on the concepts of two childhood games mashed into one, Dots and Boxes (https://en.wikipedia.org/wiki/Dots_and_boxes) and Tic-Tac-Toe (<https://en.wikipedia.org/wiki/Tic-tac-toe>). The game board is laid out in a three-by-three grid. Each player will take a turn placing a stick along the side of a box, between two dots, to earn one point. If a stick completes a box, then the player takes ownership of the box, earning five points. The game is won when a player owns three boxes in a row, horizontally, vertically, or diagonally. If no player can own three boxes in a row, the winner of the game is determined by the player with the highest score.

To keep the app and the service side relatively simple, we will eliminate a lot of state management. When the player opens the app, they will have to connect to the game service. They will have to provide a gamer tag, or username, and an email address. Optionally, they can upload a picture of themselves to use as a profile picture.

Once connected, the player will then see a list of all the other players connected to the same game service; this is called the Lobby. The player's status of either **Ready to play** or **In a match** is displayed along with the player's gamer tag and profile picture. If the player is not in a match, a button is available to challenge the player to a match.

Challenging a player to a match will cause the app to prompt the opponent to respond to the challenge, either accept or decline it. If the opponent accepts the challenge, then both players are navigated to a new game board where the player who received the challenge will have the first turn. Both players' statuses will update to **In a match** in all the other players' lobbies.

Play will alternate between players as they choose a location to place a single stick. Each time a stick is placed by a player, the game board and score will update on both players' devices. When a stick is placed that completes one or more squares, the player then wins that square, and a pile of stones is placed in the center of the square. When all sticks have been placed, or a player owns three stones in a row, the game is over, the players navigate back to the Lobby, and their status is updated to "Ready to play."

If a player leaves the app during a game, then they will have forfeited the match and the remaining opponent will be credited with the win and navigated back to the Lobby.

Now that we understand what we want to build, let's get down to the details.

We recommend that you use the same solution we used in *Chapter 9, Setting Up a Backend for a Game Using Azure Services*, because this will make code sharing easier. If you don't want to go through all of *Chapter 9*, you can get the completed source from *Chapter 9*, at

<https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition/tree/chapters/nine/main>.

We will build this app in four sections:

- **Services** – All the classes that are needed to connect and interact with the Azure Functions backend that was built in *Chapter 9, Setting Up a Backend for a Game Using Azure Services*.
- **Connect page** – This will consist of the view and view model needed to allow a user to connect to the game server as a player.
- **Lobby page** – The Lobby is where the player can send and receive challenges with other players. In this section, we will build the view and view model for the lobby.
- **Game page** – This is where players can take turns playing a game of *Sticks and Stones*. In this section, we will build the view and view model needed to make that happen.

Let's start by creating the project for the .NET MAUI app.

Building the game app

It's time to start building the app. Open the **SticksAndStones** solution from the previous chapter and follow these steps to create the project:

1. Open the **Create a new project** wizard by selecting **File, Add**, then **New Project...** from the Visual Studio menu:

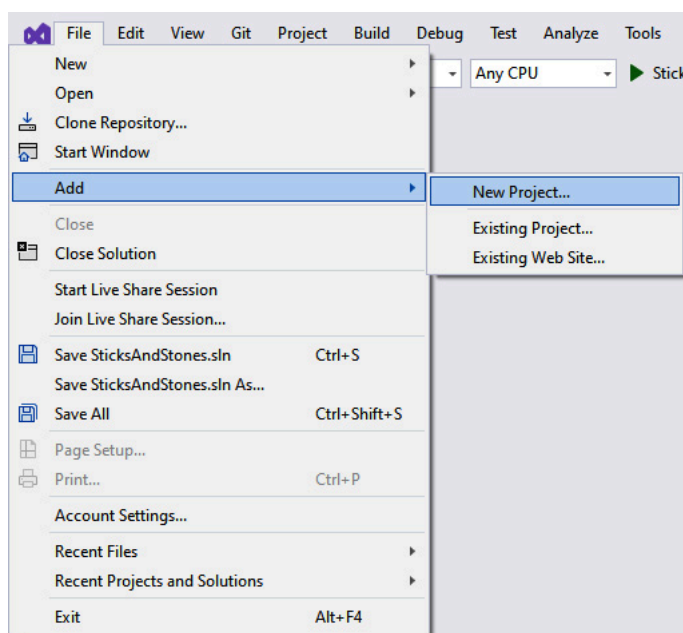


Figure 10.1 – File | Add | New Project...

- In the search field, type **maui** and select the **.NET MAUI App** item from the list, or select it from **Recent project templates** if it is listed:

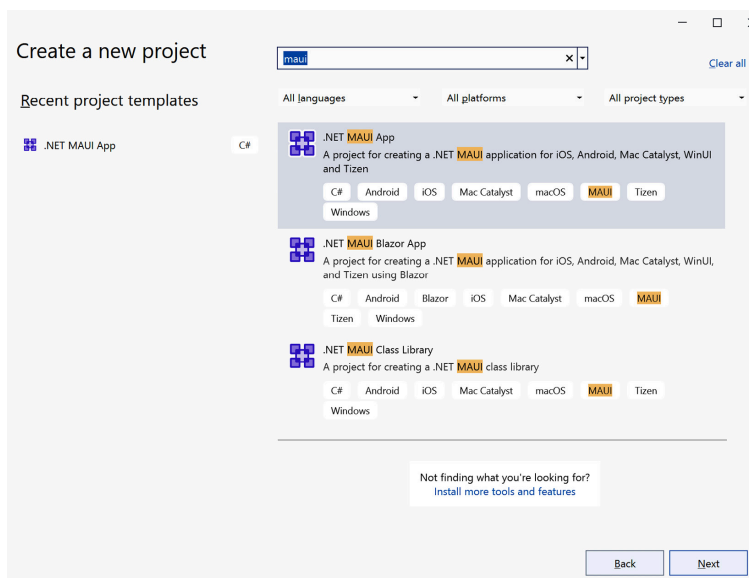


Figure 10.2 – Create a new project

- Click **Next**.
- Enter **SticksAndStones.App** as the name of the app and, under **Solution**, select **Add to solution**, as shown in the following screenshot:

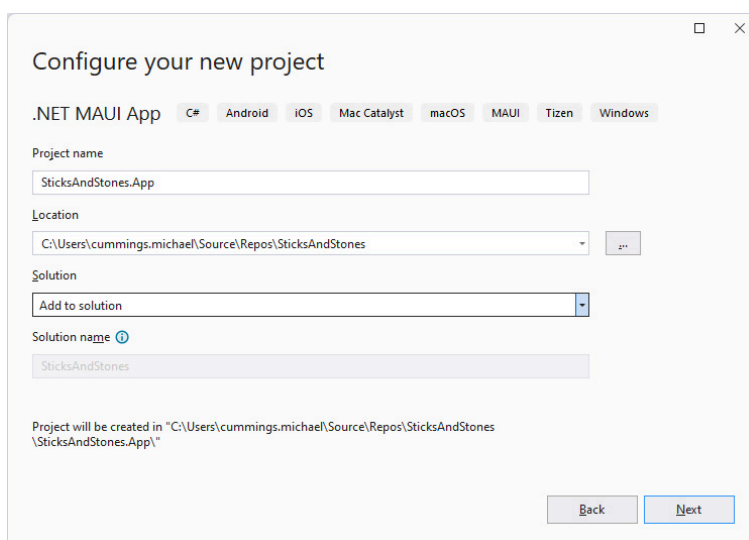


Figure 10.3 – Configure your new project

- Click **Next**.
- The last step will prompt you for the version of .NET Core to support. At the time of writing, .NET 6 is available as **Long-Term Support (LTS)**, and .NET 7 is available as **Standard Term Support**. For the purposes of this book, we will assume that you will be using .NET 7:

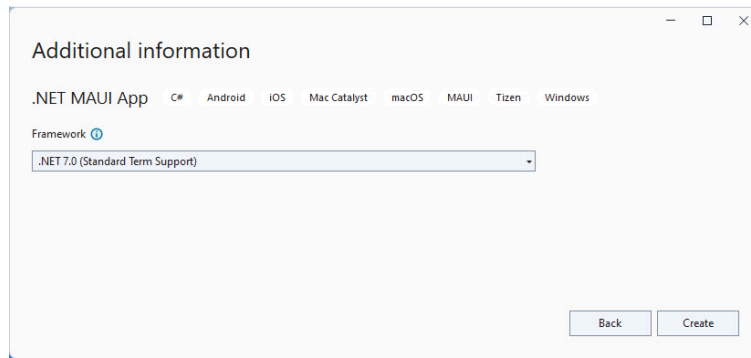


Figure 10.4 – Additional information

7. Finalize the setup by clicking **Create** and wait for Visual Studio to create the project.

Now we have created the .NET MAUI project for our game screens, let's configure it so that it's ready to add the services and views. We will need to add a project reference to the **SticksAndStones.Shared** project, as well as a few NuGet packages. Follow these steps to complete the setup of the **SticksAndStones.App** project:

1. Right-click the **SticksAndStones.App** project in **Solution Explorer** and select **Properties**.
2. In the **Properties** window, use the search box at the top to search for **Default namespace**.
3. Change the **Default namespace** property value to `$(MSBuildProjectName.Split(".")[0].Replace(" ", "_"))`.
This will split the project name on ".", using only the first part and replacing any spaces with underscores.
4. Add a NuGet Package reference to **CommunityToolkit.Mvvm** as, in other chapters, we will be using this package to simplify the implementation of data bindings to properties and commands.
5. Add a NuGet Package reference to **CommunityToolkit.Maui**. We will be using the **GravatarImageSource** class from this package to render an avatar for the user. For .NET 7, you will need to use version 6.1.0 of the NuGet package. 7.0+ has .NET 8 as a dependency.
6. Open the **MauiProgram.cs** file and add the highlighted line shown here:

```
using CommunityToolkit.Maui;
using Microsoft.Extensions.Logging;
namespace SticksAndStones.App
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureFonts(fonts =>
                {
                    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
                }
            );
        }
    }
}
```

```

        })
        .UseMauiCommunityToolkit();

    #if DEBUG
        builder.Logging.AddDebug();
    #endif

    return builder.Build();
}
}
}

```

This will configure **CommunityToolkit** for use within the app.

7. Add a NuGet package reference to **Microsoft.Extensions.Logging.Abstractions**. This package is used to log messages from the Azure Functions functions for debugging.
8. Add a NuGet package reference to **Microsoft.Extensions.Logging.Debugging**. This package is used to log messages from the Azure Functions functions for debugging.
9. Add a NuGet package reference to **Microsoft.AspNetCore.SignalR.Client**. This package is required for the app to connect to the SignalR Hub we created in *Chapter 9* and receive messages.
10. Add a Project reference to the **SticksAndStones.Shared** project. This will give us access to the messages and objects we created in *Chapter 9*.

That's it for project creation. Next, we will start with creating classes that interact directly with our service.

Creating the game services

The first thing we will do is create a service that will be used to communicate with the Azure Functions functions service created in *Chapter 9*, *Setting Up a Backend for a Game Using Azure Services*. The service will be broken down into three main classes:

- **GameService** – Methods and properties for calling the Azure Functions and receiving SignalR messages.
- **ServiceConnection** – Holds the references to **HttpClient** and SignalR Hub instances. Also provides methods for safely making calls using **HttpClient**.
- **Settings** – Stores and retrieves the URL for the server used by **HttpClient**. It also stores the connection details provided by the user.

We will start with the **Settings** class since both **GameService** and **ServiceConnection** will depend on **Settings**.

Creating the Settings service

The **Settings** service is used to store values that are needed between app runs. It will use the .NET MAUI **Preferences** class to store these values in a cross-platform manner. Use the following steps to implement the **Settings** class:

1. In the **SticksAndStones.App** project, create a new folder named **Services**.
2. In the newly created **Services** folder, create a new class named **Settings**.
3. Make the class public.
4. Create a **const string** field named **LastPlayerKey** and initialize it like so:

```
private const string LastPlayerKey = nameof(LastPlayerKey);
```

5. Create a **const string** field named **ServerUrlKey** and initialize it like this:

```
private const string ServerUrlKey = nameof(ServerUrlKey);
```

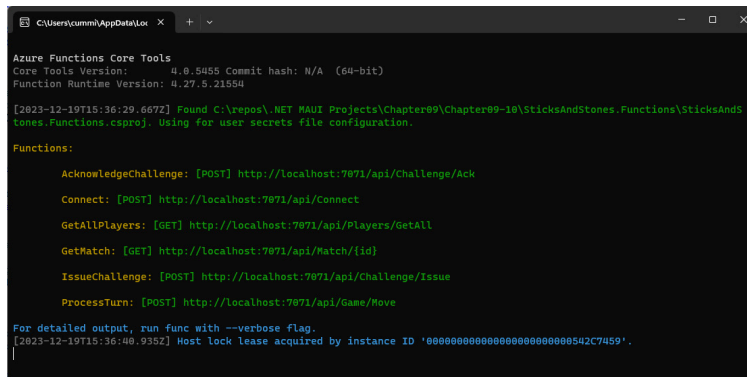
These two fields are used by the .NET MAUI **Preferences** class to store the values for the server URL and the login details for the last time the user logged in.

6. Add a **private const string** field named **ServerUrlDefault** as follows:

```
#if DEBUG && ANDROID
    private const string ServerUrlDefault = "http://10.0.2.2:7071/api";
#else
    private const string ServerUrlDefault = "http://localhost:7071/api";
#endif
```

This snippet uses the conditional compilation features of C# to switch the **ServerUrlDefault** value for Android devices. The **10.0.2.2** IP address is a special value used by the Android emulators to be able to access the host computer's **localhost** address. This is very useful when testing the app using the **Azurite** development environment for Azure Functions.

You may need to adjust the port number, highlighted in the preceding listing, for your specific development environment. Azure Functions will display the server URL when started from Visual Studio, as shown in the following screenshot:



```
C:\Users\cummh\AppData\Local\AzureFunctionsCoreTools\
Azure Functions Core Tools
Core Tools Version: 4.0.5455 Commit hash: N/A (64-bit)
Function Runtime Version: 4.27.5.21554

[2023-12-19T15:36:29.667Z] Found C:\repos\NET MAUI Projects\Chapter09\Chapter09-10\SticksAndStones.Functions\SticksAndStones.Functions.csproj. Using for user secrets file configuration.

Functions:

    AcknowledgeChallenge: [POST] http://localhost:7071/api/Challenge/Ack
    Connect: [POST] http://localhost:7071/api/Connect
    GetAllPlayers: [GET] http://localhost:7071/api/Players/GetAll
    GetMatch: [GET] http://localhost:7071/api/Match/{id}
    IssueChallenge: [POST] http://localhost:7071/api/Challenge/Issue
    ProcessTurn: [POST] http://localhost:7071/api/Game/Move

For detailed output, run func with --verbose flag.
[2023-12-19T15:36:40.935Z] Host lock lease acquired by instance ID '00000000000000000000000000000000542c7459'.
```

Figure 10.5 – Azure Functions console output

USING THE AZURE FUNCTIONS HOSTED IN AZURE

If you followed the steps in the Chapter 9 section called *Deploying the functions to Azure*, then you can use the URL for the Azure Function App cre-

ated in Chapter 9, in the *Creating the Azure service for functions* section. The URL is displayed on the **Overview** tab of the Azure Functions App.

7. Now, add a **public string** property named **ServerUrl** with the following implementation:

```
public string ServerUrl
{
    get => Preferences.ContainsKey(ServerUrlKey) ?
        Preferences.Get(ServerUrlKey, ServerUrlDefault) :
        ServerUrlDefault;
    set => Preferences.Set(ServerUrlKey, value);
}
```

This code will get the server URL from the **Preferences** store if it is present; if not, it will use the **serverUrlDefault** value. The property will store the new value in the **Preferences** store.

8. Add the following **using** declarations at the top of the **Settings.cs** file:

```
using SticksAndStones.Models;
using System.Text.Json;
```

This will enable us to use our model and the **JsonSerializer** classes.

9. Create a new property named **LastPlayer** that is of the **Player** type, as follows:

```
public Player LastPlayer
{
    get
    {
        if (Preferences.ContainsKey>LastPlayerKey))
        {
            var playerJson = Preferences.Get>LastPlayerKey, string.Empty);
            return JsonSerializer.Deserialize<Player>(playerJson, new JsonSerializerOptions(Json
        )
        return new();
    }
    set => Preferences.Set>LastPlayerKey, JsonSerializer.Serialize(value, new JsonSerializerOpt
}
```

Here, the property **set** method will convert the **Player** object to a **Json** string before storing it in **Preferences** and, when getting the property, if it exists in the **Preferences** store, convert the stored **Json** to a **Player** object. If there is no value in the **Preferences** store, then the **get** method will return an empty **Player** object.

10. The final step for the **Settings** class is to register it with the dependency injection container. Open the **MauiProgram.cs** file in the **SticksAndStones.App** project, then add the following highlighted code to the **CreateMauiApp** method:

```
public static MauiApp CreateMauiApp()
{
    var builder = MauiApp.CreateBuilder();
    builder
        .UseMauiApp<App>()
```



```

.ConfigureFonts(fonts =>
{
    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
});
#if DEBUG
    builder.Logging.AddDebug();
#endif
    builder.Services.AddSingleton<Services.Settings>();
    return builder.Build();
}

```

With the **Settings** class complete, we can now focus on the **ServiceConnection** class.

Creating the ServiceConnection class

The **ServiceConnection** class encapsulates the functionality needed to communicate with the Azure Functions service. It has methods to call the function methods and return the results, with appropriate error handling. It is also responsible for initializing the SignalR Hub instance that is used for real-time communication. The **ServiceConnection** class has a couple of dependencies that we need, so let's put them together first.

The first thing to add is logging. Having logging during debugging can be very helpful in figuring out problems, especially when dealing with asynchronous processes. Communicating with Azure Functions will have a lot of asynchronous operations. To enable logging while debugging, add the highlighted code to the **CreateMauiApp** method in the **MauiProgram** class in the **SticksAndStones.App** project:

```

#if DEBUG
    builder.Logging.AddDebug();
    builder.Services.AddLogging(configure =>
    {
        configure.AddDebug();
    });
#endif
    builder.Services.AddSingleton<Services.Settings>();
    return builder.Build();

```

This will add an instance of **ILoggingProvider** to the services container. The **ILoggerProvider** instance will provide instances of **ILogger<T>**. This will enable the use of **ILogger<T>** as a dependency in the **ServiceConnection** class constructor.

MORE ON LOGGING PROVIDERS

Read more about how logging providers work, and logging in general, at <https://learn.microsoft.com/en-us/dotnet/core/extensions/logging-providers>.

Now, when making requests to APIs using HTTP, it is a common and good practice to use asynchronous calls so that you do not block the main or UI thread. All UI updates, such as animation, button clicks, taps on the

screen, or text changes, happen on the UI thread. HTTP calls can take a non-trivial amount of time to complete, which can cause the app to become unresponsive to a user.

Error handling in asynchronous programming can be difficult. To help with errors when making API calls, we are going to use a couple of classes to encapsulate the exceptions; these classes are **AsyncError** and **AsyncExceptionError**. We need **AsyncError** and **AsyncExceptionError** because it is a bad practice to serialize and deserialize any class instances that derive from **System.Exception**. Not all classes derived from **System.Exception** are serializable, and even if they are, you may not be able to deserialize them due to a missing type – for example, the type is available on the server but not on the client. Create a new file named **AsyncError.cs** in the **SticksAndStones.App** project and replace the contents with the following code:

```
using System.Text.Json.Serialization;
namespace SticksAndStones;
public record AsyncError
{
    [JsonPropertyName("message")]
    public string Message { get; set; }
}
public record AsyncExceptionError : AsyncError
{
    [JsonPropertyName("innerException")]
    public string InnerException { get; set; }
}
```

The **AsyncError** class has a single property, **Message**. The **Message** property is decorated with the **JsonPropertyName** attribute so that it can be serialized if needed, using a lowercase version of the property name. **AsyncExceptionError** inherits from **AsyncError** and adds an additional property, **InnerException**. The **InnerException** property is also attributed with **JsonPropertyName**.

The last class we will need is **AsyncLazy<T>**. You may have already used **Lazy<T>** in other applications you have written. It's very handy when you want to delay the creation of a class until right before you need it. If you never need it, it doesn't get created. But **Lazy<T>** does not work great with asynchronous programming, so if you wanted to lazy instantiate a class that is created asynchronously, that becomes tedious. Luckily for us, Stephen Toub, who works for Microsoft on the .NET team, created **AsyncLazy<T>**. To add it to the **SticksAndStones.App** project, create a new file named **AsyncLazy~1.cs** and replace the contents with the following:

```
using System.Runtime.CompilerServices;
namespace SticksAndStones;
// AsyncLazy<T>, Microsoft, Stephen Toub, .NET Parallel Programming Blog, https://devblogs.microso
public class AsyncLazy<T> : Lazy<Task<T>>
{
    public AsyncLazy(Func<T> valueFactory) :
        base(() => Task.Factory.StartNew(valueFactory))
```

```

    { }
    public AsyncLazy(Func<Task<T>> taskFactory) :
        base(() => Task.Factory.StartNew(() => taskFactory()).Unwrap())
    { }
    public TaskAwaiter<T> GetAwaiter() { return Value.GetAwaiter(); }
}

```

LEARN MORE ABOUT ASYNCLAZY<T>

Visit the .NET blog to learn more about how Stephen Toub created the `AsyncLazy<T>` class:

<https://devblogs.microsoft.com/pfxteam/asynclazyt/>.

That completes the changes needed to start implementing the `ServiceConnection` class. To create the class, follow these steps:

1. Create a new class named `ServiceConnection` in the `SticksAndStones.App` project in the `Services` folder. Change the class to **public sealed** and inherit from `IDisposable`:

```
public sealed class ServiceConnection : IDisposable
```

2. Modify the namespace declarations at the top of the file to the following:

```

using Microsoft.AspNetCore.Http.Connections.Client;
using Microsoft.AspNetCore.SignalR.Client;
using Microsoft.Extensions.Logging;
using SticksAndStones.Models;
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;

```

These are needed to reference the types needed in the following steps.

3. Add the following **private** fields to the class:

```

private readonly ILogger log;
private readonly HttpClient httpClient;
private readonly JsonSerializerOptions serializerOptions;

```

The `serializerOptions` is used to make sure the JSON that is sent and received from the Azure Functions functions can be serialized and deserialized properly.

4. Now, add a **public** property named `Hub`. The type for `Hub` is `AsyncLazy<HubConnection>`. `HubConnection` is the type from the SignalR client library that is used to receive messages from the SignalR service. The property should look like the following:

```
public AsyncLazy<HubConnection> Hub { get; private set; }
```

`HubConnection` is initialized in the `ConnectHub` method. But first, let's add the constructor.

5. The constructor for the `ServiceConnection` class has two parameters: `ILogger <ServiceConnection>` and a `Settings` parameter. In the body of the constructor, the **private** fields created in *step 3* are initialized as follows:

```

public ServiceConnection(ILogger<ServiceConnection> logger, Settings settings)
{
    httpClient = new()
    {
        BaseAddress = new Uri(settings.ServerUrl)
    };
    httpClient.DefaultRequestHeaders.Accept.Add(new("application/json"));
    serializerOptions = new JsonSerializerOptions(JsonSerializerDefaults.Web);
    log = logger;
}

```

The **logger** and **settings** parameters are provided by the .NET MAUI dependency injection service. The **httpClient** field is initialized and its **BaseAddress** is assigned the settings **ServerUrl** property as a URI. Then, **DefaultHeaders** is modified to indicate to the server that the results are expected to be in JSON format. The **serializerOptions** instance is initialized to the defaults for **Web**, which is consistent with the formatting used by Azure Functions. Finally, the **log** field is initialized with the **logger** parameter value.

6. Now, let's implement the **Dispose** method. It will clean up any values that will potentially hold on to any native resources, such as networks, file handles, and so on. The two values that this class has references to that need to be disposed of are **httpClient** and **Hub**. Note that we will not have to call **Dispose** ourselves as the .NET MAUI dependency injection system will do that. Add the following code to the **ServiceConnection** class:

```

public void Dispose()
{
    httpClient?.Dispose();
    Hub?.Value?.Dispose();
    GC.SuppressFinalize(this);
}

```

7. Now, add the class to dependency injection by adding the following highlighted line of code to the **MauiProgram.cs** file:

```

builder.Services.AddSingleton<Services.Settings>();
builder.Services.AddSingleton<Services.ServiceConnection>();
return builder.Build();

```

8. Initializing the **Hub** property will happen in the **ConnectHub** method. The configuration for the SignalR SignalR Hub connection is returned to the app in the **Connect** function result. Since we haven't and won't make that call before this class is constructed, we can't create the **Hub** in the constructor. The configuration is needed before you can initialize the **Hub** instance. The **ConnectHub** method has a single parameter of **ConnectionInfo**. Add the method using the following code snippet:

```

public void ConnectHub(ConnectionInfo config)
{
    Hub = new(async () =>
    {
        var connectionBuilder = new HubConnectionBuilder();
        connectionBuilder.WithUrl(config.Url, (HttpConnectionOptions obj) =>

```

```

    {
        obj.AccessTokenProvider = async () => await Task.FromResult(config.AccessToken);
    });
    connectionBuilder.WithAutomaticReconnect();
    var hub = connectionBuilder.Build();
    await hub.StartAsync();
    return hub;
}
}

```

This method initializes the **Hub** property to a new **AsyncLazy<HubConnection>** instance. The constructor for **AsyncLazy<T>** takes **Func<T>**, which is provided using the anonymous method syntax. The anonymous method is also decorated as an **async** method, meaning that it will contain an awaited method call. The anonymous method takes no parameters and, in the body, starts by creating a new **HubConnectionBuilder**. Then, the **WithUrl** extension method is called on **HubConnectionBuilder** to set the URL for the SignalR service and provide the **AccessToken** value needed to make the connection. **AccessTokenProvider** is a **Task<string>** so the **config.AccessToken** is provided through another **async** anonymous function. The **WithAutomaticReconnect** method sets **HubConnection** instance to automatically try reconnecting the SignalR service if the connection is lost. If **WithAutomaticReconnect** isn't called, then the app is responsible for reconnecting if the connection is lost. The **HubConnection** instance is created by calling **HubConnectionBuilder.Build**. The **Hub** instance is then started with **StartAsync**, which is awaited, and then the **Hub** is returned. The thing to remember here is that when **ConnectHub** is called, the anonymous function isn't executed. The method won't be called until the first time a property or method of the **Hub** property is accessed.

The **ServiceConnection** class contains two helper functions that are used from the **GameService** class to make HTTP requests to the Azure Functions service. The first, **GetAsync<T>**, takes two parameters: a URL and a dictionary of query parameters to pass along with the URL. It returns an instance, **T**, and **AsyncError** as a **Tuple**. The **GetAsync** method will use the **GET** HTTP method when making the HTTP request. The other helper method, **PostAsync<T>**, uses the **POST** HTTP method and accepts two parameters: a URL, and an object to send in the body of the request formatted as JSON. It will return an instance of **T** from the response.

The **GetAsync<T>** and **PostAsync<T>** use a couple of helper methods; use the following code snippet to add them to the **ServiceConnection** class:

```

UriBuilder GetUriBuilder(Uri uri, Dictionary<string, string> parameters)
=> new(uri)
{
    Query = string.Join("&",
        parameters.Select(kvp =>
            $"{kvp.Key}={kvp.Value}"))
};
async ValueTask<AsyncError?> GetError(HttpResponseMessage responseMessage, Stream content)

```

```

{
    AsyncError? error;
    if (responseMessage.StatusCode == HttpStatusCode.Unauthorized)
    {
        log.LogError("Unauthorized request {@Uri}", responseMessage.RequestMessage?.RequestUri);
        return new()
        {
            Message = "Unauthorized request."
        };
    }
    try
    {
        error = await JsonSerializer.DeserializeAsync<AsyncError>(content, serializerOptions);
    }
    catch (Exception e)
    {
        error = new AsyncExceptionError()
        {
            Message = e.Message,
            InnerException = e.InnerException?.Message,
        };
    }
    log.LogError("{@Error} {@Message} for {@Uri}", responseMessage.StatusCode, error?.Message, res);
    return error;
}

```

The **GetUriBuilder** method will return a new **UriBuilder** from the provided URL and **Dictionary** of key-value pairs to use in the query string.

The **GetError** method will return either an **AsyncError** object or **AsyncExceptionError** object based on the status code or the contents of the response from the HTTP method call.

Now, we can add the **GetAsync<T>** method to the **ServiceConnection** class using the following code:

```

public async Task<(T Result, AsyncError Exception)> GetAsync<T>(Uri uri, Dictionary<string, string>
{
    var builder = GetUriBuilder(uri, parameters);
    var fullUri = builder.ToString();
    log.LogDebug("{@ObjectType} Get REST call {@RestUrl}", typeof(T).Name, fullUri);
    try
    {
        var responseMessage = await httpClient.GetAsync(fullUri);
        log.LogDebug("Response {@ResponseCode} for {@RestUrl}", responseMessage.StatusCode, fullUri);
        if (responseMessage.IsSuccessStatusCode)
        {
            try
            {
                var content = await responseMessage.Content.ReadFromJsonAsync<T>();
                log.LogDebug("Object of type {@ObjectType} parsed for {@RestUrl}", typeof(T).Name,
                    return (content, null);
            }
            catch (Exception e)
            {
                log.LogError("Error {@ErrorMessage} for when parsing ${@ObjectType} for {@RestUrl}"
                    return (default, new AsyncExceptionError()
                    {
                        InnerException = e.InnerException?.Message,

```

```

        Message = e.Message
    });
}
}
log.LogDebug("Returning error for @{RestUrl}", fullUri);
return (default, await GetError(responseMessage, await responseMessage.Content.ReadAsStrea
}
catch (Exception e)
{
    log.LogError("Error @{ErrorMessage} for REST call ${ResUrl}", e.Message, fullUri);
    // The service might not be happy with us, we might have connection issues etc..
    return (default, new AsyncExceptionError()
    {
        InnerException = e.InnerException?.Message,
        Message = e.Message
    });
}
}
}

```

While this method is a little long, what it is doing is not all that complicated. First, it uses the **GetUriBuilder** method to create the **UriBuilder** instance and build the **fullUri** string value from that. Then, it makes an HTTP **GET** call using the **HttpClient** instance to the URL. If there is any failure, the exception handler will catch it and return an **AsyncExceptionError**. If there are no errors in making the request and the response code indicates success, then the result is processed and returned. Otherwise, the result content is read for an error, and if it is found, it is returned. When the **GetAsync<T>** method returns, it will always return two items: the response of the **T** type and **AsyncError**. If either one of them isn't present, then their default value is returned or **null**.

Review and add the following code snippet to the **ServiceConnection** class to implement the **PostAsync<T>** method:

```

public async Task<(T Result, AsyncError Exception)> PostAsync<T>(Uri uri, object parameter)
{
    log.LogDebug("{@ObjectType} Post REST call @{RestUrl}", typeof(T).Name, uri);
    try
    {
        var responseMessage = await httpClient.PostAsJsonAsync(uri, parameter, serializerOptions);
        log.LogDebug("Response @{ResponseCode} for {@RestUrl}", responseMessage.StatusCode, uri);
        await using var content = await responseMessage.Content.ReadAsStreamAsync();
        if (responseMessage.IsSuccessStatusCode)
        {
            if(string.IsNullOrEmpty(await responseMessage.Content.ReadAsStringAsync()))
                return (default, null);
            try
            {
                log.LogDebug("Parse {@ObjectType} SUCCESS for {@RestUrl}", typeof(T).Name, uri);
                var result = await responseMessage.Content.ReadFromJsonAsync<T>();
                log.LogDebug("Object of type {@ObjectType} parsed for {@RestUrl}", typeof(T).Name,
                    return (result, null);
            }
        }
        catch (Exception e)
        {
            log.LogError("Error @{ErrorMessage} for when parsing ${ObjectType} for {@RestUrl}"

```

```

        return (default, new AsyncExceptionError()
        {
            InnerException = e.InnerException?.Message,
            Message = e.Message
        });
    }
}
log.LogDebug("Returning error for @{RestUrl}", uri);
return (default, await GetError(responseMessage, content));
}
catch (Exception e)
{
    log.LogError("Error @{ErrorMessage} for REST call ${ResUrl}", e.Message, uri);
    // The service might not be happy with us, we might have connection issues etc..
    return (default, new AsyncExceptionError()
    {
        InnerException = e.InnerException?.Message,
        Message = e.Message
    });
}
}

```

This method is mostly the same as **GetAsync<T>** with a couple of minor changes. First, it does not need to call **GetUriBuilder** to add the parameters to the **Uri QueryString**, as the parameters are sent as part of the request body. Second, it uses the HTTP **POST** method instead of **GET**. With those changes, much of the method is error handling to make sure that we return the right data.

And that completes the **ServiceConnection** class. **ServiceConnection** and the **Settings** service classes will be used in the next section, where we create the **GameService** class.

Creating the GameService class

The **GameService** class is a layer between the UI and the network. It uses the **ServiceConnection** class, which handles the specific network calls to create the logic we need to interact with the Azure Functions functions. For each of the functions that we created in *Chapter 9*, there is a corresponding method in the **GameService** class to make the call to the function and return the result, if any.

Follow these steps to create and initialize the class:

1. Create a new class named **GameService** in the **SticksAndStones.App** project under the **Services** folder.
2. Change the class definition to **public sealed** and inherit from the **IDisposable** interface:

```
public sealed class GameService : IDisposable
```

3. Add the following namespace declarations to the top of the file:

```
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.Messaging;
using Microsoft.AspNetCore.SignalR.Client;
```



```
using SticksAndStones.Messages;
using SticksAndStones.Models;
```

4. The **GameService** class will depend on both the **Settings** service and the **ServiceConnection** service, so we need to add them to the constructor and store the references in class fields, as shown here:

```
private readonly ServiceConnection service;
private readonly Settings settings;
public GameService(Settings settings, ServiceConnection service)
{
    this.service = service;
    this.settings = settings;
}
```

Add the preceding code snippet to the **GameService** class. .NET MAUI will provide the **Settings** and **ServiceConnection** instances through dependency injection.

5. Implement the **IDisposable** interface by adding the following method to the **GameService** class:

```
public void Dispose()
{
    service.Dispose();
    GC.SuppressFinalize(this);
}
```

6. Now, add the class to dependency injection by adding the following highlighted line of code to the **MauiProgram.cs** file:

```
#if DEBUG
    builder.Logging.AddDebug();
#endif

builder.Services.AddSingleton<Services.Settings>();
builder.Services.AddSingleton<Services.ServiceConnection>();
builder.Services.AddSingleton<Services.GameService>();
return builder.Build();
```

We will start with the **Connect** method. **Connect** will accept a **Player** object to connect as and return an updated **Player** object. Additionally, if the connection is successful, it will configure the SignalR Hub. To create the **Connect** function, follow these steps:

1. Create a **private** field of **SemaphoreSlim** called **semaphoreSlim** and initialize the field with a new instance with an initial and maximum count of **1**:

```
public sealed class GameService : IDisposable
{
    private readonly SemaphoreSlim semaphoreSlim = new(1, 1);
    private readonly ServiceConnection service;
```

The **SemaphoreSlim** class is a great way to limit the number of threads performing an operation at a time. In our case, we only want one thread making the network calls at a time. It will be used in all the methods that make network calls from the **GameService** class.

2. **GameService** will track the current player in a **public** property called **CurrentPlayer**; add the property to the class using the following code:

```
private readonly Settings settings;
public Player CurrentPlayer { get; private set; } = new Player() { Id = Guid.Empty, GameId = Gu
```

The property is initialized to an empty **Player** object.

3. Once the user has connected as a player, we will also need somewhere to store the list of online players. To do that, add the following property to the **GameService** class:

```
public ObservableCollection<Player> Players { get; } = new();
```

4. The **GameService** class also tracks the current status of the connection in a property called **IsConnected**; add the property using the following code snippet to the **GameService** class:

```
public bool IsConnected { get; private set; }
```

5. Add a **public async** method named **Connect** to the **GameService** class. It should return **Task<Player>** and take a single **Player** as a parameter, as shown:

```
public async Task<Player> Connect(Player player)
{
}
```

6. Within the **Connect** method, the first step is to make sure there is only one thread operating in the method at a time:

```
await semaphoreSlim.WaitAsync();
```

This uses the **async/await** structures in C# to create a lock that only releases when there are enough open slots in **SemaphoreSlim**. Since the **SemaphoreSlim** instance was only initialized with a single slot, only one thread can process the **Connect** method at a time.

7. To make sure the **SemaphoreSlim** instance releases the slot, we need to add exception handling around the rest of the method and call **Release** at the end. Add the following code snippet to the **Connect** method:

```
try
{
}
finally
{
    semaphoreSlim.Release();
}
return CurrentPlayer;
```

The **try/finally** block ensures that we will always call **Release** at the conclusion of the method, which will prevent the **SemaphoreSlim** instance from being starved, preventing any additional thread from entering the method. Lastly, we return the value of **CurrentPlayer**, which we will set next within the **try** block.

THERE IS ANOTHER WAY TO HANDLE SEMAPHORESLIM

Using a **try/catch/finally** block works, but it is a little clunky if you handle all your exceptions properly, or don't have any. Tom Dupont has published a helper class on his blog that allows you to use a **using** statement to manage the lifetime of the **SemaphoreSlim** instance. You can read his post at <http://www.tomdupont.net/2016/03/how-to-release-semaphore-with-using.html>. Here is an example of using his extension:

```
using var handle = semaphoreSlim.UseWaitAsync();
```

8. Within the **try** block, add the following lines of code:

```
CurrentPlayer = player;
var (response, error) = await service.PostAsync<ConnectResponse>(new($"{settings.ServerUrl}/Con
if (error is null)
{
    service.ConnectHub(response.ConnectionInfo);
    response.Players.ForEach(Players.Add);
    CurrentPlayer = response.Player;
    IsConnected = true;
}
else
{
    WeakReferenceMessenger.Default.Send<ServiceError>(new(error));
}
```

This block of code handles the call to the **Connect** function in the Azure Functions service. We start by setting the passed-in player details as the **CurrentPlayer** property. Then, the **player** instance is packaged into a **ConnectRequest** object and we pass that into a call to **PostAsync<T>** on the **ServiceConnection** instance. The URL is created from the **ServerUrl** property stored in the **Settings** service concatenated with **/Connect**. The response is expected to be of the **ConnectResponse** type and we store that in **response**. If we do not get any error, then we can call **ConnectHub** on the **ServiceConnection** instance, populate our **Players** collection, and set the **CurrentPlayer** property to the returned **Player** instance, which will have additional details from the server. If anything goes awry, then the error object will be populated, and we will send a message that contains that error to the UI.

ServiceError is the first message that we need to send to **ViewModel** instances from the **GameService** class. It is used to send errors from the **ServiceConnection** instance to **ViewModel** instances. We will add the **ServiceError** class in the next steps.

1. In the **SticksAndStones.App** project, create a new folder named **Messages**.
2. Create a new class named **ServiceError** in the **Messages** folder of the **SticksAndStones.App** project.
3. The **ServiceError** message is a simple wrapper around the **AyncError** object that can be used to send a message back to a view model. Replace the contents of the **ServiceError.cs** file with the following:

```
using CommunityToolkit.Mvvm.Messaging.Messages;
namespace SticksAndStones.Messages;
internal class ServiceError : ValueChangedMessage<AsyncError>
{
    public ServiceError(AsyncError error) : base(error)
    {
    }
}
```

4. Finally, since we are using **SemaphoreSlim** and it can hold onto native resources, we should make sure that those are released properly. Add the following highlighted code to the **Dispose** method to clean up the **semaphoreSlim** field:

```
public void Dispose()
{
    semaphoreSlim.Release();
    semaphoreSlim.Dispose();
    service.Dispose();
    GC.SuppressFinalize(this);
}
```

This concludes the **Connect** method for now.

The next three methods are called from the **Lobby** page. The first method is used to refresh the list of players. It is called when the user pulls down the list causing a refresh, or if the SignalR Hub is reconnected. To implement the **RefreshPlayerList** method, follow these steps:

1. the **RefreshPlayerList** method takes no arguments and returns **Task**; add the method to the **GameService** class as follows:

```
public async Task RefreshPlayerList()
{
    await semaphoreSlim.WaitAsync();
    try
    {
        var getAllPlayers = service.GetAsync<GetAllPlayersResponse>(new($"{settings.ServerUrl}/
        var (response, error) = await getAllPlayers;
        if (error is null)
        {
            Players.Clear();
            response.Players.ForEach(Players.Add);
        }
        else
        {
            WeakReferenceMessenger.Default.Send<ServiceError>(new(error));
        }
    }
    finally
    {
        semaphoreSlim.Release();
    }
}
```

2. To refresh the list of players when the SignalR Hub reconnects, add the following highlighted code to the **Connect** method:

```

if (error is null)
{
    service.ConnectHub(response.ConnectionInfo);
    response.Players.ForEach(Players.Add);
    CurrentPlayer = response.Player;
    (await service.Hub).Reconnected += (s) => { return RefreshPlayerList(); };
}

```

This line of code is interesting. First, we await **service.Hub**, then set the **Reconnected** event to an anonymous function that calls **RefreshPlayerList**. If you recall, the **Hub** property in the **ServiceConnection** class is **AsyncLazy<T>**. The first time we reference the **Hub** property, it will initialize itself, asynchronously, hence the **await** call.

The next method that is used from the **Lobby** page is **IssueChallenge**. The **IssueChallenge** method is called from the **Lobby** page when a player wishes to play a match against another player. The **IssueChallenge** method does not return any value since the actual response to the challenge will come back through the SignalR Hub. The method will send the request to the server and handle any errors, as shown here:

```

public async Task IssueChallenge(Player opponent)
{
    await semaphoreSlim.WaitAsync();
    try
    {
        var (response, error) = await service.PostAsync<IssueChallengeResponse>(new($"{settings.Se
        if (error is not null)
        {
            WeakReferenceMessenger.Default.Send<ServiceError>(new(error));
        }
    }
    finally
    {
        semaphoreSlim.Release();
    }
}

```

Add the preceding code to the **GameService** class. The **SendChallengeResponse** method, which is called when the opposing player responds to a challenge, is very similar to the **IssueChallenge** method, as shown:

```

public async Task SendChallengeResponse(Guid challengeId, Models.ChallengeResponse challengeRespon
{
    await semaphoreSlim.WaitAsync();
    try
    {
        var (response, error) = await service.PostAsync<string>(new($"{settings.ServerUrl}/Challen
        if (error is not null)
        {
            WeakReferenceMessenger.Default.Send<ServiceError>(new(error));
        }
    }
    finally

```

```

    {
        semaphoreSlim.Release();
    }
}

```

Add the **SendChallengeResponse** method to the **GameService** class. That completes the methods needed to support the **Lobby** page. The final page in our app is the **Game** page. There are three more methods that are needed by the **Game** page. Follow these steps to add them:

1. Add the **EndTurn** method, which will send the player's move to the **Game** server, using the following code snippet:

```

public async Task<(Game?, string?)> EndTurn(Guid gameId, int position)
{
    await semaphoreSlim.WaitAsync();
    try
    {
        var (response, error) = await service.PostAsync<ProcessTurnResponse>(new($"{settings.Se
        if (error is not null)
        {
            return (null, error.Message);
        }
        else return (response.Game, null);
    }
    finally
    {
        semaphoreSlim.Release();
    }
}

```

EndTurn is very similar to the **IssueChallenge** and **SendChallengeResponse** methods, with a minor exception: it returns the updated **Game** object and an error message if present.

2. the **GetPlayerId** method is a small helper function to search the **Players** list and return the **Player** instance that matches the ID passed in. Use the following code snippet to add the **GetPlayerById** method:

```

public Player? GetPlayerById(Guid playerId)
{
    if (playerId == CurrentPlayer.Id)
        return CurrentPlayer;
    return (from p in Players where p.Id == playerId select p).FirstOrDefault();
}

```

3. the **GetMatchById** method is the last method that will make a call to the backend. In this case, it will retrieve a **Match** object given an ID. Using the following code snippet, add **GetMatchById** to the **GameService** class:

```

public async Task<Match> GetMatchById(Guid matchId)
{
    await semaphoreSlim.WaitAsync();
    try
    {
        var (response, error) = await service.GetAsync<GetMatchResponse>(new($"{settings.Server
        if (error != null) { }
    }
}

```

```

        if (response.Match != null)
            return response.Match;
        return new Match();
    }
    finally
    {
        semaphoreSlim.Release();
    }
}

```

The final piece to the **GameService** class is the handling of the events received through the SignalR Hub. To refresh our memory from *Chapter 9*, the backend functions will send the following events to the clients via SignalR:

- **PlayerUpdatedEventArgs**
- **ChallengeEventArgs**
- **GameStartedEventArgs**
- **GameUpdatedEventArgs**

We will handle each of these events in the **GameService** method. To implement the handlers for these events, follow these steps:

1. When the **Hub** receives **PlayerUpdatedEventArgs**, we will need to update **Player** in the **Players** collection with the new values. We will create a helper function to handle that work, as follows:

```

private void PlayerStatusChangedHandler(PlayerUpdatedEventArgs args)
{
    var changedPlayer = (from player in Players
                        where player.Id == args.Player.Id
                        select player).FirstOrDefault();
    if (changedPlayer is not null)
    {
        changedPlayer.MatchId = args.Player.MatchId;
    }
    else if (args.Player.Id != CurrentPlayer.Id)
    {
        Players.Add(args.Player);
    }
}

```

The **PlayerStatusChangedHandler** method will locate the changed player in the **Players** collection and update the relevant fields of the instance or add it if it doesn't exist.

2. To call the **PlayerStatusUpdateHandler** class when the **PlayerUpdated** event is received, add the following highlighted code to the **Connect** method:

```

if (error is null)
{
    service.ConnectHub(response.ConnectionInfo);
    response.Players.ForEach(Players.Add);
    CurrentPlayer = response.Player;
    IsConnected = true;
    (await service.Hub).On<PlayerUpdatedEventArgs>(Constants.Events.PlayerUpdated, PlayerStatus

```

```
(await service.Hub).Reconnected += (s) => { return RefreshPlayerList(); };
}
```

The other three events will send messages to the **ViewModel** instance using **WeakReferenceManager**. First, we need to add the message types, using the following steps:

1. Add a new class named **ChallengeReceived** to the **Messages** folder in the **SticksAndStones.App** project.
2. Replace the contents of the **ChallengeReceived.cs** file with the following:

```
using CommunityToolkit.Mvvm.Messaging.Messages;
using SticksAndStones.Models;
namespace SticksAndStones.Messages;
public class ChallengeReceived : ValueChangedMessage<Player>
{
    public Guid Id { get; init; }
    public ChallengeReceived(Guid id, Player challenger) : base(challenger)
    {
        Id = id;
    }
}
```

3. Add a new class named **MatchStarted** in the **Messages** folder of the **SticksAndStones.App** project.
4. Replace the contents of the **MatchStarted.cs** file with the following code:

```
using CommunityToolkit.Mvvm.Messaging.Messages;
using SticksAndStones.Models;
namespace SticksAndStones.Messages;
public class MatchStarted : ValueChangedMessage<Match>
{
    public MatchStarted(Match match) : base(match)
    {
    }
}
```

5. Add a new class named **MatchUpdated** to the **SticksAndStones.App** project in the **Messages** folder.
6. Replace the contents of the **MatchUpdated.cs** file with the following code:

```
using CommunityToolkit.Mvvm.Messaging.Messages;
using SticksAndStones.Models;
namespace SticksAndStones.Messages;
class MatchUpdated : ValueChangedMessage<Match>
{
    public MatchUpdated(Match match) : base(match)
    {
    }
}
```

7. To send a message when the event is received, add the following highlighted code to the **Connect** method in the **GameService** class:


```

service.ConnectHub(response.ConnectionInfo);
response.Players.ForEach(Players.Add);
CurrentPlayer = response.Player;
IsConnected = true;
(await service.Hub).On<PlayerUpdatedEventArgs>(Constants.Events.PlayerUpdated, PlayerStatusChan
(await service.Hub).On<ChallengeEventArgs>(Constants.Events.Challenge, (args) => WeakReferenceM
(await service.Hub).On<MatchStartedEventArgs>(Constants.Events.MatchStarted, (args) => WeakRefe
(await service.Hub).On<MatchUpdatedEventArgs>(Constants.Events.MatchUpdated, (args) => WeakRefe
(await service.Hub).Reconnected += (s) => { return RefreshPlayerList(); }

```

That concludes the **GameService** class. We have all the methods needed to interact with the backend functions and we are handling the events that are being sent to the clients. The next portion of the chapter will add the pages needed to present the screens to the user, starting with the **Connect** page.

Creating the Connect page

The **Connect** page, as shown in *Figure 10.6*, is the first screen a user is presented with after the app loads. The page contains four main elements: an entry box for the player's gamer tag, an entry box for the player's email address, an image control for the player's avatar, and the **Connect** button.

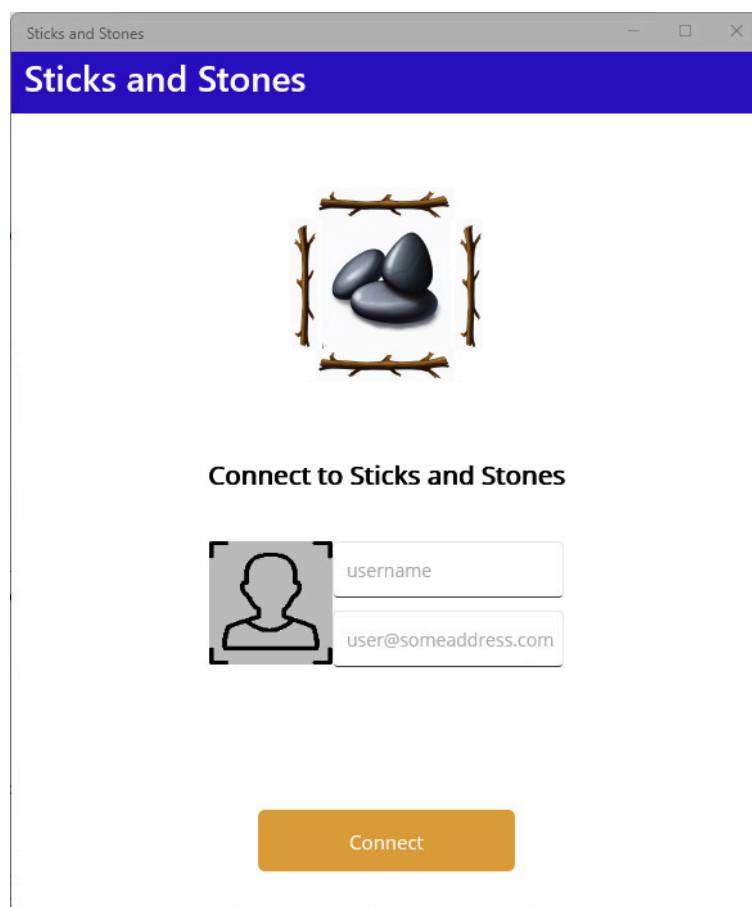


Figure 10.6 – The Connect page

The **Connect** page will consist of several parts:

- A **ViewModel** file called **ConnectViewModel.cs**
- A XAML file called **ConnectView.xaml**, which contains the layout
- A code-behind file called **ConnectView.xaml.cs**, which will carry out the data-binding process
- A XAML file containing the layout for a custom button control, called **ActivityButton.xaml**
- The code-behind for **ActivityButton** in **ActivityButton.xaml.cs**

We will begin the implementation of the **Connect** page by implementing **ConnectViewModel** first.

Adding ConnectViewModel

ConnectViewModel – along with **LobbyViewModel** and **GameView** model – will inherit from a single base class called **ViewModelBase**. The **ViewModelBase** class provides the necessary functionality to implement refreshing the page. Not all pages will use this feature, but it will be available. To add **ViewModelBase**, follow these steps:

1. Create a new folder named **ViewModels** in the **SticksAndStones.App** project.
2. Add a new class named **ViewModelBase** in the **ViewModels** folder.
3. Add the following namespace declarations at the top of the **ViewModelBase.cs** file:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
```

4. Change the class declaration to **public abstract partial** and derive the class from **ObservableRecipient**:

```
public abstract partial class ViewModelBase : ObservableRecipient
```

ObservableRecipient comes from **CommunityToolkit**. If you have worked through the other chapters in this book, you will have seen view models that derive from **ObservableObject**, which implements **INotifyPropertyChanged**. **ObservableRecipient** extends **ObservableObject** and adds built-in support for working with implementations of the .NET MAUI **IMessage** interface. To learn more about **ObservableRecipient**, visit <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/observablerecipient>.

5. Add a **private bool** field named **canRefresh** with the **ObservableProperty** attribute:

```
[ObservableProperty]
private bool canRefresh;
```

6. Add a **private bool** field named **isRefreshing** with the **ObservableProperty** attribute:

```
[ObservableProperty]
private bool isRefreshing;
```

7. Add a **private** method named **CanExecuteRefresh** that takes no parameters and returns a **bool**. The method signature and implementation are in the following code snippet:

```
private bool CanExecuteRefresh() => CanRefresh && !IsRefreshing;
```

8. Add a new **protected virtual** method named **RefreshInternal** that returns a **Task** and its implementation returns **Task.CompletedTask**, as shown here:

```
protected virtual Task RefreshInternal() => Task.CompletedTask;
```

9. Add the **Refresh** method as shown here:

```
[RelayCommand(CanExecute = nameof(CanExecuteRefresh))]
public async Task Refresh()
{
    IsRefreshing = true;
    await RefreshInternal();
    IsRefreshing = false;
    return;
}
```

The **Refresh** method is a **Command**, meaning that it can be bound to XAML elements as **RefreshCommand**. The **CanExecuteRefresh** method is used to determine the enabled/disabled state for the command. The command itself flips the **IsRefreshing** Boolean and calls the **RefreshInternal** method where classes derived from **ViewModelBase** would put the specific implementation.

Now that **ViewModelBase** has been implemented, we can implement **ConnectViewModel**. The **ConnectViewModel** class has bindable properties for the player's gamer tag and email address and various states for commands. Finally, there is a command to establish a connection to the game services. Let's implement the **ConnectViewModel** class by following these steps:

1. Create a new class named **ConnectViewModel** in the **ViewModels** folder.
2. Alter the class definition to **public partial** and derive from **ViewModelBase**, as shown here:

```
public partial class ConnectViewModel : ViewModelBase
{
}
```

3. **ConnectViewModel** depends on **GameService** and the **Settings** service so let's add a constructor to acquire them through dependency injection and **private** fields to store their values, as shown here:

```
public partial class ConnectViewModel : ViewModelBase
{
    private readonly GameService gameService;
    private readonly Settings settings;
    public ConnectViewModel(GameService gameService, Settings settings)
    {
```

```

        this.gameService = gameService;
        this.settings = settings;
    }
}

```

4. To use the **GameService** and **Settings** classes, you'll need to add a namespace declaration to the top of the file:

```
using SticksAndStones.Services;
```

5. Add a **private string** field named **gamerTag** attributed with **ObservableProperty** to make it bindable, as shown in the following code snippet:

```
[ObservableProperty]
private string gamerTag;
```

6. Add a **private string** field named **emailAddress** attributed with **ObservableProperty** to make it bindable, as shown in this code snippet:

```
[ObservableProperty]
private string emailAddress;
```

7. In the constructor for **ConnectViewModel**, initialize the bindable properties from the last time the user connected, as shown in the following snippet:

```

{
    this.gameService = gameService;
    this.settings = settings;
    // Load Player settings
    var player = settings.LastPlayer;
    Username = player.GamerTag;
    EmailAddress = player.EmailAddress;
}

```

8. The **Connect** page does not need to refresh the view, so disable that functionality by adding the following line of code to the beginning of the constructor:

```
CanRefresh = false;
```

9. To implement the **Connect** command, we will need four things: a **string** indicating the status of the command, a **bool** to indicate the current state of the command, a method to return if the command is enabled, and the method for the command itself. To add the status as a string, add the following code above the constructor in the **ConnectViewModel** class:

```
[ObservableProperty]
private string connectStatus;
```

We mark this field with **ObservableProperty** so that it is bindable into the view.

10. To add the **isConnecting** field to track the state of the command, add the following code under the **connectStatus** field:

```
[ObservableProperty]
private bool isConnecting;
```

11. The **CanExecuteConnect** method will return **true** if the command is enabled, and **false** if not. Add the method using the following code snippet under the **isConnecting** field:

```
private bool CanExecuteConnect() => !string.IsNullOrEmpty(GamerTag) && !string.IsNullOrEmpty(Em
```

12. The **Connect** command will call a **Connect** method to establish the connection with the game server. This is mostly just to keep the methods small and manageable. Add the following private **Connect** method to the **ConnectViewModel** class:

```
private async Task<Player> Connect(Player player)
{
    // Get SignalR Connection
    var playerUpdate = await gameService.Connect(player);
    if (gameService.IsConnected)
    {
        // If the player has an in progress match, take them to it.
        if (gameService.CurrentPlayer?.MatchId != Guid.Empty)
        {
            await Shell.Current.GoToAsync($"///Match", new Dictionary<string, object>() { { "Ma
        }
        else
        {
            await Shell.Current.GoToAsync($"///Lobby");
        }
    }
    return playerUpdate;
}
```

This method will call the **Connect** method on the **GameService** class passing in the player details. If the connection is successful, then the user is navigated to the Lobby page, unless they are currently in a game, in which case, they are navigated to the Game page.

NAVIGATION IN .NET MAUI SHELL

In .NET MAUI, navigation is performed by calling **GoToAsync** from the **Shell** object. The **Shell** object can be obtained by either casting **App.Current.MainPage** to a **Shell** object, or by using the **Shell.Current** property. The route passed to **GoToAsync** can be either relative to the current location or absolute. The valid forms of relative and absolute routes are as follows:

- **route** – The route will be searched for upward from the current position and, if found, pushed onto the navigation stack
- **/route** – The route will be searched for downward from the current position and, if found, pushed onto the navigation stack
- **//route** – The route will be searched for upward from the current position and, if found, will replace the navigation stack

- `///route` – The route will be searched for downward from the current position and, if found, will replace the navigation stack

To learn more about routes and navigation, visit

<https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/shell/navigation>.

13. Add the method implementing the **Connect** command to the bottom of the **Connect ViewModel** class using the following code snippet:

```
[RelayCommand(CanExecute = nameof(CanExecuteConnect))]
public async Task Connect()
{
    IsConnecting = true;
    ConnectStatus = "Connecting...";
    var player = settings.LastPlayer;
    player.GamerTag = GamerTag;
    player.EmailAddress = EmailAddress;
    player.Id = (await Connect(player)).Id;
    settings.LastPlayer = player;
    ConnectStatus = "Connect";
    IsConnecting = false;
}
```

The command is very straightforward. It sets the **IsConnecting** and **ConnectStatus** properties, then updates the **Player** values from the view. Then, it calls **Connect**, passing in the current **Player** instance. The ID of the returned player is captured and set back on **LastPlayer** in **Settings**. Finally, the **ConnectStatus** and **IsConnecting** properties are set back to their defaults.

14. To wrap things up, we need to add a couple of attributes to make sure that properties are updated appropriately as values change. For instance, when the **IsConnecting** property is changed, we need to ensure that the **CanExecuteConnect** method is evaluated again. To do this, we add the **NotifyCanExecuteChangeFor** attribute to the **IsConnecting** field, as shown here:

```
[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof(ConnectCommand))]
private bool isConnecting;
```

Since the **gamerTag** field and **emailAddress** field are also referenced in the **CanExecuteConnect** method, we should add the attribute to those fields as well, as shown here:

```
[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof(ConnectCommand))]
private string gamerTag;
[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof(ConnectCommand))]
private string emailAddress;
```

We have nearly completed **ConnectViewModel**. The final feature to implement is the handling of messages that we may receive from **GameService**. The **ObservableObject** implementation from **CommunityToolkit** provides

a feature to make subscribing and unsubscribing to these messages clean.

To implement the message handlers, follow these steps:

1. Add a new **private** method named **OnServiceError** to the **ConnectViewModel** class, using the following code snippet:

```
private void OnServiceError(AsyncError error)
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        await Shell.Current.CurrentPage.DisplayAlert("There is a problem...", error.Message, "O
    });
}
```

2. We will subscribe to the **ServiceError** messages from the **OnActivated** event method of the **ObservableObject** class. Add the following method to the **ConnectViewModel** class to subscribe to the **ServiceError** message:

```
protected override void OnActivated() => Messenger.Register<ServiceError>(this, (r, m) => OnSer
```

3. To unsubscribe from the **ServiceError** messages, add the following method to the **ConnectViewModel** class:

```
protected override void OnDeactivated() => Messenger.Unregister<ServiceError>(this);
```

4. In the constructor for **ConnectViewModel**, we need to enable the **OnActivated** and **OnDeactivated** events that are raised by **ObservableObject**. These events are the recommended places to subscribe and unsubscribe to messages. Add the following line of code to the end of the constructor to enable the events:

```
IsActive = true;
```

Setting **IsActive** to **true** will cause the **OnActivated** event to fire.

Setting it to **false** will cause the **OnDeactivated** event to fire.

5. To have the **OnDeactivated** event fire, we need to set **IsActive** to **false**. In the **Connect** method, add the highlighted line of code:

```
private async Task<Player> Connect(Player player)
{
    // Get SignalR Connection
    var playerUpdate = await gameService.Connect(player);
    if (gameService.IsConnected)
    {
        IsActive = false;
        // If the player has an in progress match, take them to it.
        if (gameService.CurrentPlayer?.MatchId != Guid.Empty)
        {
            await Shell.Current.GoToAsync($"///Match", new Dictionary<string, object>() { { "Ma
        }
        else
        {
            await Shell.Current.GoToAsync($"///Lobby");
        }
    }
}
```

```
        return playerUpdate;
    }
}
```

6. Open the **MauiProgram.cs** file in the **SticksAndStones.App** project and add the following highlighted line to register **ConnectViewModel** with dependency injection:

```
builder.Services.AddSingleton<Services.GameService>();
builder.Services.AddTransient<ViewModels.ConnectViewModel>();
return builder.Build();
```

This concludes the implementation of **ConnectViewModel**. The **ConnectViewModel** class controls the entry of the user's gamer tag and email. It connects the user to the game server using their player details.

Adding the Connect view

The **Connect** view looks fairly simple, but there is a lot to it. We will break down the creation of the view into three sections:

- Creating the **ActivityButton** control:
The **ActivityButton** control is the button used to initiate a connection to the backend services. While a simple button might do the trick, what if we had an animation that indicated the **connect** operation was in progress and the text of the button updated as well? That is what **ActivityButton** will do, in a reusable control.
- Creating the images:
There are a few images used on this page. All of them were generated using AI. We'll explore how that was done.
- Building the view:
This is where we bring **ActivityButton** together with our custom images and the built-in controls of .NET MAUI to make **ConnectView** appear as it does in the figures.

Let's start by building the **ActivityButton** control.

Creating the ActivityButton control

So, what is this **ActivityButton** control? It's basically a button with an **ActivityIndicator** that will only show up while the task behind the button is doing its work. The complexity of this control comes from the fact that we are making a general-purpose control instead of a specialized control. So, for all intents and purposes, it needs to act like a normal **Button** and **ActivityIndicator**. We are only going to implement the feature that we need for this application, but even then, it's still a reusable control.

From **Button**, we want to have the following XAML attributes:

- **Text**, **FontFamily**, and **FontSize**
- **Command** and **CommandParameter**

From **ActivityIndicator**, we will have **IsRunning**.

Each of these XAML elements will be bindable, like their original properties. An example of what the XAML might look like for declaring this control as an element would be as follows:

```
<controls:ActivityButton IsRunning="{Binding IsConnecting}"
    Text="{Binding ConnectStatus}"
    BackgroundColor="#e8bc65"
    Command="{Binding ConnectCommand}"
    HorizontalOptions="Center"
    WidthRequest="200"
    HeightRequest="48"/>
```

This listing comes from the actual XAML we will be creating for **ConnectView.xaml** later in this section.

The attributes that are copied from the two underlying controls need to be able to bind to the view model. This requires that they are implemented as bound properties. To create a bound property, you need two things: a property and a **BindableProperty** that references the property. The **BindableProperty** provides the functionality to keep the view model property, which implements **INotifyPropertyChanged**, with the property of the control. Let's create the **Command** bindable property as an example:

1. Create a new folder named **Controls** in the **SticksAndStones.App** project.
2. Add a new .NET MAUI **ContentView** (XAML) called **ActivityButton**.
3. Open the **ActivityButton.xaml.cs** file.
4. Create a new **public ICommand** property named **Command**, as shown in the following listing:

```
public ICommand Command
{
    get => (ICommand)GetValue(CommandProperty);
    set { SetValue(CommandProperty, value); }
}
```

BindableProperty properties have a circular reference with the properties they are bound to, so you will get red squiggles until we complete the next step. This looks like almost every other property we have created, except that the **get** and **set** methods are just delegating to the **GetValue** and **SetValue** methods, respectively. **GetValue** and **SetValue** are provided by the **BindableObject** class, which **ContentView** ultimately inherits from. **GetValue** and **SetValue** are the **BindableObject** equivalents to **INotifyPropertyChanged** for view models. Calling them not only stores the values but also sends notifications that the value has changed.

5. Now, add the **BindableProperty** property for the **Command** property, using the following code snippet:

```
public static readonly BindableProperty CommandProperty = BindableProperty.Create(
    propertyName: nameof(Command),
    returnType: typeof(ICommand),
```

```
declaringType: typeof(ActivityButton),
defaultBindingMode: BindingMode.TwoWay);
```

CommandProperty is of the **BindableProperty** type and is created by using the **Create** factory method of the **BindableProperty** class. We pass in the name of the property we are binding to (**Command**), the type that property returns (**Icommand**), the declaring type (which is **ActivityButton** in this case), and then what mode we want the binding to have. There are four options for **BindingMode**:

1. **OneWay** – The default, propagates changes from the source (the view model) to the target (the control)
2. **OneWayToSource** – This is the reverse of **OneWay**, propagating changes from the target (the control) to the source (the view model)
3. **TwoWay** – This propagates changes in both directions
4. **OneTime** – This propagates the changes only when **BindingContext** changes and all **INotifyPropertyChanged** events are ignored

These two pieces – the normal property that you would use in most of your C# classes, and **BindableProperty** – provide the complete functionality we need to create the custom control.

Now that we understand how to implement **BindableProperty** on a XAML control, we can complete the implementation of **ActivityButton**.

Let's start by updating the XAML and then we will follow that with the remaining **BindableProperty** implementations. The following steps will walk you through creating the control:

1. The template we chose to create the XAML and **.cs** files is not quite what we need for **ActivityButton**. We will need to alter the underlying root control from **ContentView** to **Frame**. We use **Frame** to wrap our layout with a border. Open the **ActivityButton.cs** file and update the class definition to inherit from **Frame** instead of **ContentView**, as follows:

```
public partial class ActivityButton : Frame
```

2. Now, open the **ActivityButton.xaml** file and modify it to look like the following:

```
<Frame xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        x:Class="SticksAndStones.Controls.ActivityButton">
</Frame>
```

In addition to changing **ContentView** to **Frame**, also remove the contents of **Frame** as we won't be reusing it.

3. Let's name our control to make it easier to reference it later. Typically, in C#, if you want to reference the instance of the class, you will use the **this** keyword. That doesn't exist by default in XAML so add the **x:Name** attribute with the value of **this** to mimic C#.
4. Update the **Frame** element and add the **BackgroundColor** attribute with a value of **{x:StaticResource Primary}**. **Primary** is defined in the **Resources/Styles/Colors.xaml** file and we can reference it using the **StaticResource** extension method.

- Update the **Frame** element and add the **CornerRadius** attribute with a value of **5**. This will give our button rounded corners.
- Add the **Padding** attribute with a value of **12** to the **Frame** element. This will ensure that there is plenty of whitespace around the control. The **Frame** element should now look like the following:

```
<?xml version="1.0" encoding="utf-8" ?>
<Frame xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
      xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
      x:Class="SticksAndStones.Controls.ActivityButton"
      x:Name="this"
      BackgroundColor="{x:StaticResource Primary}"
      CornerRadius="5"
      Padding="12">

</Frame>
```

- To get **ActivityIndicator** and **Label** centered side by side within **Frame**, we will use **HorizontalStackLayout** contained in **VerticalStackLayout**. **StackLayout** controls ignore the alignment options for the direction of the control, so, for example, **VerticalStackLayout** ignores the **VerticalOptions** property of its children and **HorizontalStackLayout** ignores the **HorizontalOptions** property of its children. This is because, by its nature, **HorizontalStackLayout** is in control of laying out its children in the horizontal plane, and the same is true for **VerticalStackLayout**, except in the vertical plane. Add the following highlighted code to the XAML:

```
<Frame xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
      xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
      x:Class="SticksAndStones.Controls.ActivityButton"
      BackgroundColor="{x:StaticResource Primary}"
      CornerRadius="5"
      Padding="12">
  <VerticalStackLayout>
    <HorizontalStackLayout HorizontalOptions="CenterAndExpand" Spacing="10">
    </HorizontalStackLayout>
  </VerticalStackLayout>
</Frame>
```

- Within the **HorizontalStackLayout** element, add the following XAML:

```
<ActivityIndicator HeightRequest="15" WidthRequest="15"
  Color="{x:StaticResource White}"
  IsRunning="{Binding Source={x:Reference this},Path=IsRunning}"
  IsVisible="{Binding Source={x:Reference this},Path=IsRunning}"
  VerticalOptions="CenterAndExpand"/>
```

ActivityIndicator will have a **Height** and **Width** value of **15** and a **Color** value of **White**. The **IsRunning** and **IsVisible** properties are bound to the control's **IsRunning** property. We haven't created the **IsRunning** property yet, so this won't work until we do. The **x:Reference** markup extension allows us to bind the property to the parent control, which we named **this** in *step 3*.

- Now, we can add **Label** within **HorizontalStackLayout** using the following XAML:

```
<Label x:Name="buttonLabel" TextColor="{x:StaticResource White}"
      Text="{Binding Source={x:Reference this},Path=Text}"
      FontSize="15"
      VerticalOptions="CenterAndExpand"
      VerticalTextAlignment="Center"
      HorizontalTextAlignment="Start" />
```

10. When the user taps or clicks anywhere in **Frame**, **Command** should be run. To configure that, we will use **GestureRecognizer**.

GestureRecognizer is XAML's way of providing event handlers. There are several different kinds of **GestureRecognizer**:

1. **DragGestureRecognizer** and **DropGestureRecognizer**
2. **PanGestureRecognizer**
3. **PinchGestureRecognizer**
4. **PointerGestureRecognizer**
5. **SwipeGestureRecognizer**
6. **TapGestureRecognizer**

For **ActivityButton**, we are interested in **TapGestureRecognizer**.

Since the action to take is undefined until this control is used on a view, **TapGestureRecognizer** will invoke a command when **Frame** is tapped. Add the following XAML to the **Frame** element to create

TapGestureRecognizer:

```
<Frame.GestureRecognizers>
  <TapGestureRecognizer Command="{Binding Source={x:Reference this},Path=Command}" CommandPar
</Frame.GestureRecognizers>
```

The **Command** attribute and the **CommandParameter** attribute of **TapGestureRecognizer** are set to bind to the parent controls' **Command** and **CommandParameter** properties.

If the **IsRunning** property is **true**, then **Frame** should be disabled, and the reverse is true as well. **DataTrigger** is a XAML way of setting properties of one control based on changes in another control's properties. To add the triggers for **Frame**, add the highlighted XAML to the control:

```
<Frame.Triggers>
  <DataTrigger TargetType="Frame" Binding="{Binding Source={x:Reference this},Path=IsBusy}" V
    <Setter Property="IsEnabled" Value="False" />
  </DataTrigger>
  <DataTrigger TargetType="Frame" Binding="{Binding Source={x:Reference this},Path=IsBusy}" V
    <Setter Property="IsEnabled" Value="True" />
  </DataTrigger>
</Frame.Triggers>
```

11. That concludes the XAML portion of the control. Open the **ActivityButton.xaml.cs** file and we can add the missing properties, starting with **CommandParameter**:

```
public static readonly BindableProperty CommandParameterProperty = BindableProperty.Create(
    propertyName: nameof(CommandParameter),
    returnType: typeof(object),
    declaringType: typeof(ActivityButton),
    defaultBindingMode: BindingMode.TwoWay);
public object CommandParameter
{
    get { return (object)GetValue(CommandParameterProperty); }
    set { SetValue(CommandParameterProperty, value); }
}
```

```

    get => GetValue(CommandParameterProperty);
    set { SetValue(CommandParameterProperty, value); }
}

```

Add the previous code listing to the **ActivityButton** class. Other than the name, there isn't anything significant about this property from the **Command** property. **CommandParameter** allows you to specify parameters to pass to **Command**, but using XAML.

12. The **Label** control is populated from the **Text** property. To add the **Text** property, add the following code to the **ActivityButton** class:

```

public static readonly BindableProperty TextProperty = BindableProperty.Create(
    propertyName: nameof(Text),
    returnType: typeof(string),
    declaringType: typeof(ActivityButton),
    defaultValue: string.Empty,
    defaultBindingMode: BindingMode.TwoWay);
public string Text
{
    get => (string)GetValue(TextProperty);
    set { SetValue(TextProperty, value); }
}

```

In the case of the **Text** property, **returnType** has changed to **string**, but otherwise, it's similar to **Command** and **CommandParameter**.

13. The next property we need to implement is the **IsRunning** property, as follows:

```

public static readonly BindableProperty IsRunningProperty = BindableProperty.Create(
    propertyName: nameof(IsRunning),
    returnType: typeof(bool),
    declaringType: typeof(ActivityButton),
    defaultValue: false);
public bool IsRunning
{
    get => (bool)GetValue(IsRunningProperty);
    set { SetValue(IsRunningProperty, value); }
}

```

14. To allow the size and font of the text to be changed, we implement the **FontSize** and **FontFamily** properties:

```

public static readonly BindableProperty FontFamilyProperty = BindableProperty.Create(
    propertyName: nameof(FontFamily),
    returnType: typeof(string),
    declaringType: typeof(ActivityButton),
    defaultValue: string.Empty,
    defaultBindingMode: BindingMode.TwoWay);
public string FontFamily
{
    get => (string)GetValue(Label.FontFamilyProperty);
    set { SetValue(Label.FontFamilyProperty, value); }
}
public static readonly BindableProperty FontSizeProperty = BindableProperty.Create(
    nameof(FontSize),
    typeof(double),
    typeof(ActivityButton),
    Device.GetNamedSize(NamedSize.Small, typeof(Label)),

```

```
BindingMode.TwoWay);
public double FontSize
{
    set { SetValue(FontSizeProperty, value); }
    get { return (double)GetValue(FontSizeProperty); }
}
```

That completed **ActivityButton**. We will use **ActivityButton** in the *Creating the Connect view* section right after we create the images we need for the game.

Creating images using Bing Image Creator

There are a few images that are used in the game. They are as follows:

- A horizontal stick
- A vertical stick
- A pile of stones

Creating these images can be quite time-consuming and, based on your artistic abilities, not quite what you expected. It is quite possible that for your app you may opt to hire a graphics designer or artist to create your digital assets for you. Recently, a new option has become available, and that is to use AI to generate images. In this section, we will look at how to use **Bing Image Creator** to create the images that are needed for the game.

Bing Image Creator uses an English description of the scene that you would like to see and attempts to create it. There are a few keywords that you can use to direct the Image Creator in the artistic style of the image to create, such as *game art*, *digital art*, or *photorealistic*.

Let's get started by creating the stick image:

1. Open <https://bing.com/create> in Microsoft Edge or your favorite web browser.
2. If asked, log in with your Microsoft account. This can be the same account that you used in *Chapter 9* to log in to the Azure portal.
3. In the prompt box, type in the following prompt, then press **Create**:

```
A single wood stick, positioned horizontally, with five stubs where branches would be and no le
```

Image Creator will generate four different images based on your description. If you aren't satisfied with the result, adjust the description slightly and try again. The more descriptive you are, the better your result. Try to get a stick that is nearly vertical or horizontal since it will be easier to rotate and crop the image. It will also look much better if it is on a bright white background.

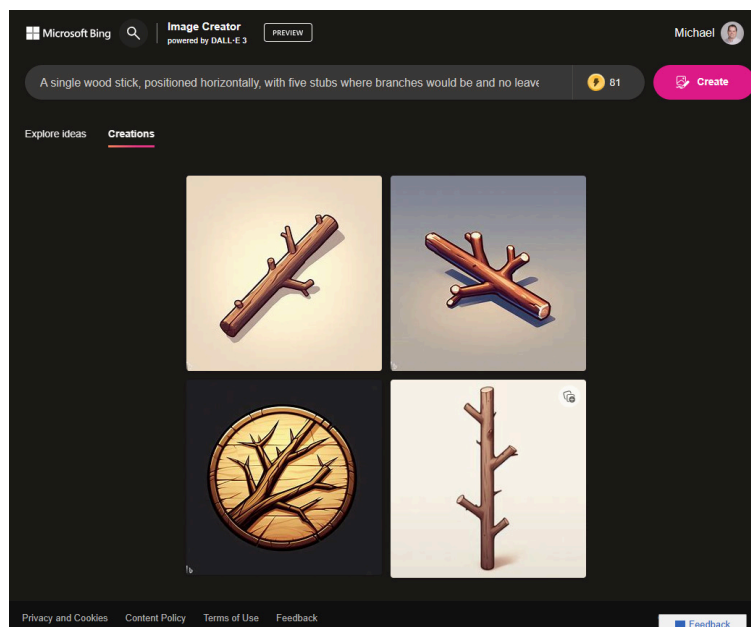


Figure 10.7 – An Image Creator sample set of images

4. Once you have an image you are satisfied with, click on the image to open it up.
5. Click the **Download** button to save the image to your local computer.
6. Now, open the downloaded file in your favorite image editor. The images created by Image Creator are roughly 1024 x 1024, and ideally, the image should be a 3:9 ratio, or around 300 x 900 pixels. Using your image editor tools, crop the image so that it is roughly 300 x 900 pixels.
7. Save the image as either **hstick.jpeg** if the stick is orientated horizontally, or **vstick.jpeg** if vertically, in the **Resources/Images** folder of the **SticksAndStones.App** project.
8. Using the same image editing tools, rotate the image 90 degrees so that it is the opposite orientation and save the image in the **Resources/Images** folder as **hstick.jpeg** if the stick is now orientated horizontally, or **vstick.jpeg** if vertically.

We have nearly half of the images that we need to create. Let's work on creating the stones next:

1. Open <https://bing.com/create> in Microsoft Edge or your favorite web browser.
2. If asked, log in with your Microsoft account. This can be the same account that you used in *Chapter 9* to log in to the Azure portal.
3. In the prompt box, type in the following prompt, then press **Create**:

3 grey stones, arranged closely together, no background, game art

4. Work the prompt to get three stones nicely piled together, preferably on a white background, as shown here:

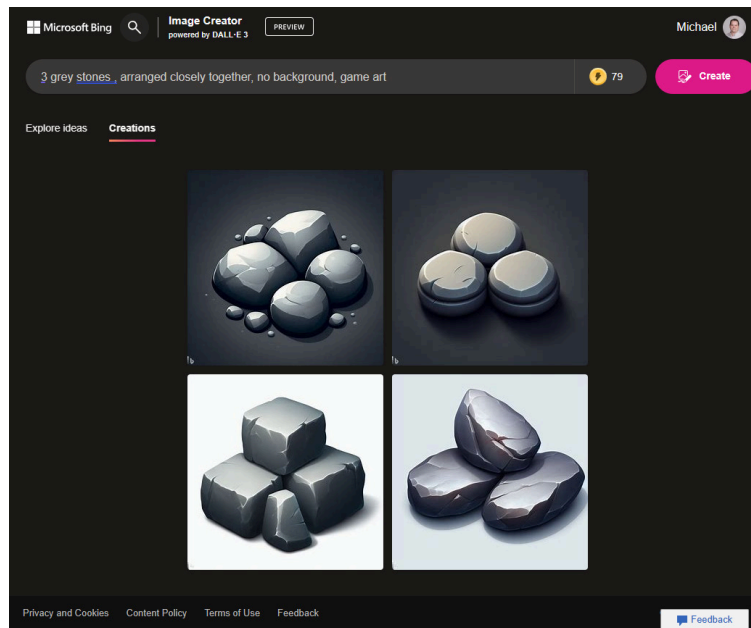


Figure 10.8 – Three stones on a white background

5. When you are satisfied with the generated image, click the image to open it.
6. Click the **Download** button to save the image to your local computer.
7. Now, open the downloaded file in your favorite image editor. Since the stones are supposed to be a square image, we can just save the file into the **Resources/Images** folder as **stones.jpeg**.

DON'T HAVE AN IMAGE EDITOR?

Don't have a favorite image editor? If you are on Windows, Paint does the job nicely, or you can use Visual Studio to edit images. On macOS, you can use Preview.

Excellent, we now have the sticks and the stones needed to play the game, and that concludes the use of Image Creator to generate our game's images. You can always go back to the site and review previous results, which is a nice feature. Now, we can move forward with creating the **Connect** view.

Creating the Connect view

The **Connect** view is the first UI other than the splash screen the user is going to see in the app. *Figure 10.6* provides a representation of what the final view will look like. The images may be different if you decide to generate your own of course. We will break this section into three parts. First, we will create the top portion of the view containing the static content, then move on to creating the middle portion of the view containing the entry controls, and then, finally, the **Connect** button. Let's get started with the top section of the view by following these steps:

1. In the **SticksAndStones.App** project, create a folder named **Views**.
2. Right-click on the **Views** folder, select **Add**, and then click **New Item....**
If you are using Visual Studio 17.7 or later, click the **Show All Templates** button in the dialog that pops up; otherwise, move to the

next step.

3. Under the **C# Items** node on the left, select **.NET MAUI**.
4. Select **.NET MAUI ContentPage (XAML)** and name it **ConnectView.xaml**.
5. Click **Add** to create the page.

Refer to the following screenshot to view the preceding information:

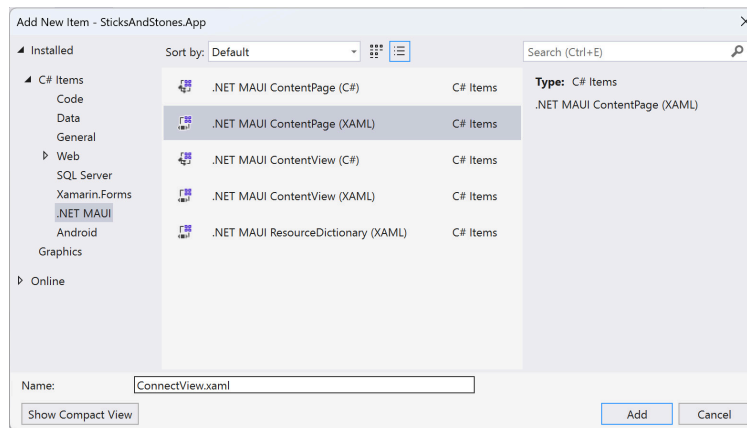


Figure 10.9 – Adding a new .NET MAUI ContentPage (XAML)

6. Change the title of the view to **Sticks & Stones**. Since XAML is a dialect of XML, the ampersand (&) must be escaped as **&** in the string.

Add the following highlighted namespaces to the **ContentView** element. They will provide us access to the classes in the **ViewModels**, **Controls**, and **Toolkit** namespaces:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:Class="SticksAndStones.Views.ConnectView"
             Title="Sticks and Stones">
```

7. To make IntelliSense happy with the bindings we will be adding, define the view model that the view is using by adding the **x:DataType** attribute to the **ContentView** element, as shown:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:Class="SticksAndStones.Views.ConnectView"
             x:DataType="viewModels:ConnectViewModel"
             Title="Sticks and Stones">
```

8. We don't want the user to use any navigation, such as the **Shell**-provided **Back** button, other than what is provided on this page, so disable it using the highlighted code in the following listing:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
```

```
xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
xmlns:controls="clr-namespace:SticksAndStones.Controls"
    xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
    x:Class="SticksAndStones.Views.ConnectView"
    x:DataType="viewModels:ConnectViewModel"
    Title="Sticks and Stones"
    NavigationPage.HasNavigationBar="False">
```

9. Finally, set **BackgroundColor** of the entire view to **White**, which will make the images blend better, by adding the following highlighted code:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
    xmlns:controls="clr-namespace:SticksAndStones.Controls"
    xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
    x:Class="SticksAndStones.Views.ConnectView"
    x:DataType="viewModels:ConnectViewModel"
    Title="Sticks & Stones"
    NavigationPage.HasNavigationBar="False"
    BackgroundColor="White">
```

10. To lay out the view, use a **Grid** control that has four rows defined; add the following code within the **ContentPage** element:

```
<Grid Margin="40">
    <Grid.RowDefinitions>
        <RowDefinition Height="8*" />
        <RowDefinition Height="2*" />
        <RowDefinition Height="8*" />
        <RowDefinition Height="1*" />
    </Grid.RowDefinitions>
</Grid>
```

Grid uses a **Margin** value of **40** to provide plenty of whitespace around the images and controls. The first row at **8** units will contain the logo for the app. The second row will contain the text **Sticks & Stones**. The third row will have the avatar image, email, and gamer tag entry controls. The final row will contain the **Connect** button. Recall that the **Height** units are relative, so row **0**, the first row, will be four times higher than row **1** and eight times higher than row **3**, the final row. The ***** symbol in the **Height** value indicates that the row can expand if needed.

11. To arrange our generated images into a box-like layout, another **Grid** control is used. Add the following listing to the view between the **</Grid.RowDefinitions>** and **</Grid>** tags:

```
<Grid Grid.Row="0" WidthRequest="150" HeightRequest="150">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="1*" />
        <ColumnDefinition Width="5*" />
        <ColumnDefinition Width="1*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="1*" />
        <RowDefinition Height="4*" />
```

```

        <RowDefinition Height="1*" />
    </Grid.RowDefinitions>
    <Image Grid.Row="0" Grid.Column="1" Source="hstick.jpeg" Aspect="Fill"/>
    <Image Grid.Row="0" Grid.Column="0" Source="vstick.jpeg" Aspect="Fill"/>
    <Image Grid.Row="1" Grid.Column="1" Source="stones.jpeg" Aspect="AspectFit"/>
    <Image Grid.Row="1" Grid.Column="2" Source="vstick.jpeg" Aspect="Fill"/>
    <Image Grid.Row="2" Grid.Column="1" Source="hstick.jpeg" Aspect="Fill"/>
</Grid>

```

This **Grid** control defines three rows and three columns. The content of **Grid** is entirely made up of **Image** controls. **Grid** is positioned in row **0** of its parent **Grid**. The children of the grid, the **Image** controls, are positioned by setting the **Grid.Row** and **Grid.Column** attributes on the **Image** control. The stick images use the **Aspect** attribute set to **Fill**. **Fill** allows the image to scale to completely fill the content area; to do so, it may not scale uniformly along both the x and y axis. The stones use an **Aspect** value of **AspectFit**. This will uniformly scale the image till at least one side fits, which may cause letterboxing. There are two more options for the **Aspect** property: **Center**, which does no scaling, and **AspectFill**, which will scale until both axes have filled the view, which may cause clipping.

12. A **Label** element containing the text **Connect to Sticks & Stones** is added to the outer **Grid** control and it's placed in row **1**, which is the second row of **Grid**. Add the following code to the outer **Grid** control after the inner **Grid** control added in *step 11*:

```

<Label Grid.Row="1" Text="Connect to Sticks & Stones" FontSize="20" TextColor="Black" FontA

```

13. The next section of the page contains the avatar image, gamer tag entry, and email entry controls. **HorizontalStackLayout** and **VerticalStackLayout** controls are used to arrange the controls. Add the following snippet to the outer **Grid** control after the **Label** control added in *step 12*:

```

    <HorizontalStackLayout Grid.Row="2" HorizontalOptions="Center">
        <VerticalStackLayout Spacing="10" >
            <Image HeightRequest="96" WidthRequest="96" BackgroundColor="LightGrey">
                <Image.Source>
                    <toolkit:GravatarImageSource
                        Email="{Binding EmailAddress}"
                        Image="MysteryPerson" />
                </Image.Source>
            </Image>
        </VerticalStackLayout>
        <VerticalStackLayout Spacing="10" >
            <Entry Placeholder="username" Keyboard="Email" Text="{Binding Username}" HorizontalText
            <Entry Placeholder="user@someaddress.com" Keyboard="Email" Text="{Binding EmailAddress}"
        </VerticalStackLayout>
    </HorizontalStackLayout>

```

The **HorizontalStackLayout** control is assigned to the **Grid** row **2**, the third row. It is also centered horizontally within the row. The first **VerticalStackLayout** arranges the controls that make up the avatar. It contains an **Image** element, whose source is set to an instance of **GravatarImageSource**. **GravatarImageSource** uses the **Gravatar** service (<https://gravatar.com>) to provide avatars when given an email

address. The **EmailAddress** property of **ConnectViewModel** is bound to the **Email** property of **GravatarImageSource**. The image will automatically update on changes to **EmailAddress**. The **Image** property uses the **MysteryPerson** value to provide a plain profile when there isn't a Gravatar available for the email address. The second **VerticalStackLayout** contains two **Entry** controls: the first, for the gamer tag, is bound to the **Username** property of **ConnectViewModel**, and the second is bound to the **EmailAddress** property of **ConnectViewModel**. The **Keyboard** attribute determines which virtual keyboard is displayed when the focus is on the control. See <https://learn.microsoft.com/en-us/dotnet/maui/user-interface/controls/entry#customize-the-keyboard> for more information on customizing the keyboard.

14. The final control to add to **ConnectView** is the **Connect** button. Use the following snippet to add the button to the view:

```
<controls:ActivityButton Grid.Row="3"
                        IsRunning="{Binding IsConnecting}"
                        Text="{Binding ConnectStatus}"
                        BackgroundColor="#e8bc65"
                        Command="{Binding ConnectCommand}"
                        HorizontalOptions="Center"
                        WidthRequest="200"
                        HeightRequest="48"/>
```

The **Connect** button uses the **ActivityButton** control created in the *Creating the ActivityButton control*. The control is positioned in row 3, the fourth row, and the **IsRunning** attribute is bound to the **ConnectViewModel.IsConnecting** method. The **Text** attribute of the button is bound to the **ConnectViewModel.ConnectStatus** property, and finally, **Command** is bound to the **ConnectViewModel.Connect** method.

15. Open the **MauiProgram.cs** file in the **SticksAndStones.App** project and add the following highlighted line to register **ConnectView** with dependency injection:

```
builder.Services.AddSingleton<Services.GameService>();
builder.Services.AddTransient<ViewModels.ConnectViewModel>();
builder.Services.AddTransient<Views.ConnectView>();
return builder.Build();
```

16. Now, we need to consume the **ConnectViewModel** instance through dependency injection and set it as the binding object. Open the **ConnectView.Xaml.cs** file and modify it as follows:

```
using SticksAndStones.ViewModels;
namespace SticksAndStones.Views;
public partial class ConnectView : ContentPage
{
    public ConnectView(ConnectViewModel viewModel)
    {
        this.BindingContext = viewModel;
        InitializeComponent();
    }
}
```

```
    }
}
```

17. Finally, we need to set **ConnectView** as the first view displayed. Open the **AppShell.xaml** file in the **SticksAndStones.App** project and update the contents of the **Shell** element as shown:

```
<Shell
    x:Class="SticksAndStones.App.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:SticksAndStones.App"
    xmlns:views="clr-namespace:SticksAndStones.Views"
    Shell.FlyoutBehavior="Disabled">
    <ShellItem Route="Connect">
        <ShellContent ContentTemplate="{DataTemplate views:ConnectView}" />
    </ShellItem>
</Shell>
```

The first of the three views in the app is complete. To test it out, follow these steps:

1. In Visual Studio, right-click the **SticksAndStones.Functions** project in **Solution Explorer**, then select **Debug | Start Without Debugging**.
2. In **Solution Explorer**, right-click the **SticksAndStones.App** project and select **Set as Startup Project**.
3. Press **F5** to start the **SticksAndStones.App** project using the debugger.

The **Connect** page is the first page our users will see when they launch the app. We have added image controls and entry controls, used data binding to bind to our view model, and extended XAML by creating a custom control and a converter. In the next section, we will add the **Lobby** page, which will allow us to start games with other players.

Creating the Lobby page

The **Lobby** page displays the list of connected players and allows a player to challenge another to a match. As additional players connect to the server, they are added to the list of available players. *Figure 10.10* shows the two views for the page, one with connected players, and the empty view when there are no additional players connected.

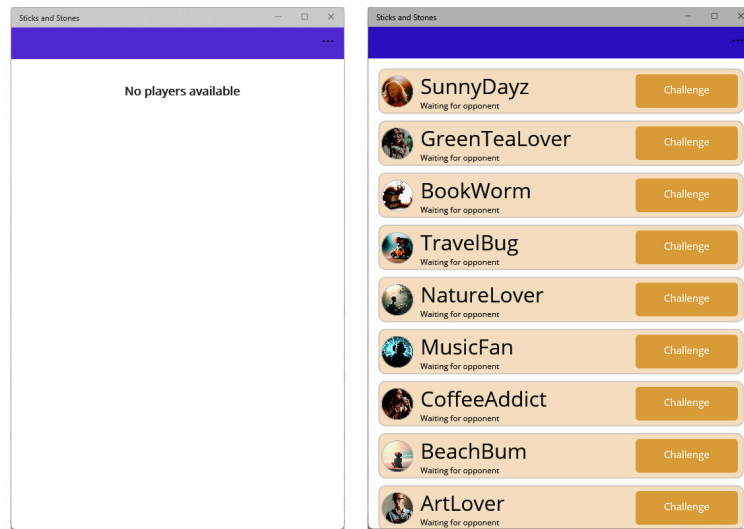


Figure 10.10 – The Lobby views

Each player is displayed on a card with their avatar image, gamer tag, status, and a button to allow the player to challenge the other to a match.

This page is comprised of two **ViewModel** classes, not one. As you might expect, there is the **LobbyViewModel** class, the **LobbyViewModel** which class has a collection of **PlayerViewModel** instances. In addition to the **ViewModel** classes, there is the **LobbyView** class. Let's get started by creating the **PlayerViewModel** class.

Adding PlayerViewModel

PlayerViewModel is very much like all our other **ViewModel** classes but with one slight difference: it isn't bound directly to a view in the same way. Otherwise, it has the same purpose: abstract the model, in this case, **Player**, away from the UI that displays it. **PlayerViewModel** provides all the needed binding properties to display each individual player card in **LobbyView**. To add **PlayerViewModel**, follow these steps:

1. In the **SticksAndStones.App** project, under the **ViewModels** folder, create a new class named **PlayerViewModel**.
2. Add the following namespaces to the top of the file:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using SticksAndStones.Models;
using SticksAndStones.Services;
```

3. Add the following code to the class:

```
private readonly Player playerModel;
private readonly GameService gameService;
public PlayerViewModel(Player player, GameService gameService)
{
    playerModel = player;
    this.gameService = gameService;
}
```

This adds two **private** fields to hold the values of the arguments passed to the constructor. As with **ConnectViewModel**, the constructor arguments are provided by dependency injection.

4. The player card displays the player's gamer tag, avatar, and status.

Add the following code to the **PlayerViewModel** class to add the **Id** and **GamerTag** properties:

```
public Guid Id => playermodel.Id;
public string GamerTag => playerModel.GamerTag;
```

For **PlayerViewModel**, some of the properties that we bind to are not implemented with **ObservablePropertyAttribute**. That is because we are providing their values from the **Player** model directly. So, the **get** method of the property just returns the corresponding property of the model object. There is no defined **set** method, so this property is essentially a one-way data binding.

5. The **Status** property is a little different since it does not exist on our **Player** model. The **Status** property is a textual indication of whether the player is in a match or not. The **Player** model does have a **MatchId** property, so if the **Player** model has a valid **MatchId** (i.e., not **Guid.Empty**), then the status would be **"In a match"**; otherwise, that status would be **"Waiting for opponent"**. Add the following code to **PlayerViewModel** to implement the **Status** property:

```
public bool IsInMatch => !(playerModel.MatchId == Guid.Empty);
public string Status => IsInMatch switch
{
    true => "In a match",
    false => "Waiting for opponent"
};
```

The **IsInMatch** property is used to simplify the **Status** property implementation. It will also be used later in the class. The **Status** property is a simple switch on **IsInMatch** and returns the proper **string** value.

6. To add a command to handle the **Challenge** button, add the following code to the **PlayerViewModel** class:

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(ChallengeStatus))]
private bool isChallenging = false;
public string ChallengeStatus => IsChallenging switch
{
    true => "Challenging...",
    false => "Challenge"
};
public bool CanChallenge => !IsInMatch && !IsChallenging;
[RelayCommand(CanExecute = nameof(CanChallenge))]
public void Challenge(PlayerViewModel opponent)
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        IsChallenging = true;
        bool answer = await Shell.Current.CurrentPage.DisplayAlert("Issue Challenge!", $" You a
        if (answer)
        {
```

```

        await gameService.IssueChallenge(opponent.Player);
    }
    IsChallenging = false;
});
return;
}

```

The command is prevented from executing while it is currently waiting for a challenge response, which makes sense – no need to nag the other player. The **IsChallenging** property is set to **true** while challenging and **false** when it is complete. The **CanChallenge** property is a combination of **IsInMatch** and **IsChallenging**, meaning that you can't challenge the same player while you have an existing challenge in progress, and you can't challenge a player who is already in a match with another player. **ChallengeStatus**, which is used as the text for the button, is bound to the **IsChallenging** value and updates when that property is updated. You may have noticed that our command takes a single parameter. This is used to operate on the correct player.

That completes **PlayerViewModel**. Next, **LobbyViewModel** is used to encapsulate the collection of **PlayerViewModel** objects.

Adding LobbyViewModel

LobbyViewModel is a fairly straightforward implementation. It has a collection of **PlayerViewModel** objects that are exposed to the UI, it allows the user to pull to refresh the view, and it handles the messages of **ChallengeReceived**, **MatchStarted**, and **ServiceError**. Follow these steps to implement **LobbyViewModel**:

1. In the **SticksAndStones.App** project, inside the **ViewModels** folder, create a new class named **LobbyViewModel**.
2. Add the following namespaces to the top of the file:

```

using System.Collections.ObjectModel;
using System.Collections.Specialized;
using CommunityToolkit.Mvvm.Messaging;
using SticksAndStones.Messages;
using SticksAndStones.Models;
using SticksAndStones.Services;

```

3. Modify the class declaration to a **public partial** class that inherits from **ViewModelBase**, as shown:

```

public partial class LobbyViewModel : ViewModelBase

```

4. Add the following code to the **LobbyViewModel** class:

```

private readonly GameService gameService;
public ObservableCollection<PlayerViewModel> Players { get; init; }
public LobbyViewModel(GameService gameService)
{
    this.gameService = gameService;
    Players = new(from p in gameService.Players
                  where p.Id != gameService.CurrentPlayer.Id
                  select new PlayerViewModel(p, gameService));
}

```



```

        CanRefresh = true;
        IsActive = true;
    }

```

LobbyViewModel receives an instance of **GameService** via dependency injection. The **gameService** instance is used to initialize the **Players** list. The **Players** property from the **GameService** class is a collection of the **Player** model, whereas **Players** in **LobbyViewModel** is an **ObservableCollection** instance of **PlayerViewModel**. We use **ObservableCollection** because it provides support for **INotifyPropertyChanged** and **INotifyCollectionChanged** when it is bound automatically. A LINQ query is used to get all the current players and add them to the **Players ObservableCollection**. **CanRefresh** from **ViewModelBase** is set to **true**, which enables **RefreshCommand**. Finally, **IsActive** is set to **true**, which enables the **OnActivated** and **OnDeactivated** events.

5. The **GameService.Players** list will be updated as players connect to the server. However, these changes do not get propagated to the **LobbyViewModel.Players** collection automatically. By implementing a handler for the **CollectionChanged** event of the **GameService.Players** property, we can then update the **LobbyViewModel.Players** collection appropriately. Add the following method to the **LobbyViewModel** class:

```

private void OnPlayersCollectionChanged(object? sender,
    NotifyCollectionChangedEventArgs e)
{
    if (e.Action == NotifyCollectionChangedAction.Add)
    {
        foreach (var player in e.NewItems.Cast<Player>())
        {
            Players.Add(new PlayerViewModel(player, gameService));
        }
    }
    else if (e.Action == NotifyCollectionChangedAction.Remove)
    {
        foreach (var player in e.OldItems.Cast<Player>())
        {
            var toRemove = Players.FirstOrDefault(p => p.Id == player.Id);
            Players.Remove(toRemove);
        }
    }
    else if (e.Action == NotifyCollectionChangedAction.Replace)
    {
    }
    else if (e.Action == NotifyCollectionChangedAction.Reset)
    {
        Players.Clear();
    }
}

```

The **OnPlayersCollectionChanged** method is an implementation of **NotifyCollectionChangedEventHandler**. It is called by the **ObservableCollection.CollectionChanged** event. The event is called whenever an item in the collection is added, removed, or updated. It is also called when the entire collection is cleared. This method handled the **NotifyCollectionChangedAction** values of **Add**, **Remove**, and **Reset**.

6. The **Players.CollectionChanged** event is assigned to the **OnPlayersCollectionChanged** method in the **OnActivated** event handler. Add the **OnActivated** and **OnDeactivated** methods using the following listing:

```
protected override void OnActivated()
{
    gameService.Players.CollectionChanged += OnPlayersCollectionChanged;
    // If the player has an in progress match, take them to it.
    if (gameService.CurrentPlayer?.MatchId != Guid.Empty)
    {
        MainThread.InvokeOnMainThreadAsync(async () =>
        {
            IsActive = false;
            await Shell.Current.GoToAsync(Constants.ArgumentNames.MatchId, new Dictionary<string, object>() { { Constants.ArgumentNames.MatchId, gameService.CurrentPlayer.MatchId } });
        });
    }
}
protected override void OnDeactivated()
{
    gameService.Players.CollectionChanged -= OnPlayersCollectionChanged;
}
```

In the **OnActivated** method, the **CollectionChanged** event is assigned to the **OnPlayersCollectionChanged** method and unassigned in the **OnDeactivated** method. In **OnActivated**, there is also a check to see whether the player is already in a match. If they are, then the app navigates to the **Match** view immediately. When navigating to the **Match** view, we send an argument for **Match**. This will be either the **MatchId** or the **Match** model.

7. Open the **Constants.cs** file in the **SticksAndStones.Shared** project to add the following code snippet to the **Constants** class:

```
public class ArgumentNames
{
    public static readonly string Match = nameof(Match);
    public static readonly string MatchId = nameof(MatchId);
}
```

8. While in the Lobby, there are three messages that need to be handled: **ChallengeReceived**, **MatchStarted**, and **ServerError**. Add the code in the following listing to add the handlers for each of these messages:

```
private void OnChallengeReceived(Guid challengeId, Player opponent)
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        bool answer = await Shell.Current.CurrentPage.DisplayAlert("You have been challenged!",
            await gameService.SendChallengeResponse(challengeId, answer ? Models.ChallengeResponse.Accept : Models.ChallengeResponse.Refuse));
    });
}
private void OnMatchStarted(Match match)
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        IsActive = false;
        await Shell.Current.GoToAsync($"///Match", new Dictionary<string, object>() { { Constants.ArgumentNames.MatchId, match.MatchId } });
    });
}
```

```

    }
    private void OnServiceError(AsyncError error)
    {
        MainThread.BeginInvokeOnMainThread(async () =>
        {
            IsActive = false;
            await Shell.Current.CurrentPage.DisplayAlert("There is a problem...", error.Message, "Ok");
        });
    }
}

```

In **OnChallengeReceived**, the user is prompted to accept or decline the challenge. Their response is then sent to the challenger via the **SendChallengeResponse** method of the **GameService** class.

OnMatchStarted will navigate the user to the **Match** view. Finally, **OnServiceError** will display the error to the user.

9. Add the following snippet to the top of the **OnActivated** method to register to receive the messages:

```

Messenger.Register<ChallengeReceived>(this, (r, m) => OnChallengeReceived(m.Id, m.Value));
Messenger.Register<MatchStarted>(this, (r, m) => OnMatchStarted(m.Value));
Messenger.Register<ServiceError>(this, (r, m) => OnServiceError(m.Value));

```

10. Add the following snippet to the end of the **OnDeactivated** method to stop receiving messages:

```

Messenger.Unregister<ChallengeReceived>(this);
Messenger.Unregister<MatchStarted>(this);
Messenger.Unregister<ServiceError>(this);

```

11. To refresh the **Players** list when the user pulls down on the list in the UI, add the following method to the **LobbyViewModel** class:

```

protected override async Task RefreshInternal()
{
    await gameService.RefreshPlayerList();
    return;
}

```

12. **LobbyViewModel** needs to be registered with dependency injection, so open the **MauiProgram.cs** file and add the following highlighted line of code:

```

builder.Services.AddTransient<ViewModels.ConnectViewModel>();
builder.Services.AddTransient<ViewModels.LobbyViewModel>();
builder.Services.AddTransient<Views.ConnectView>();

```

LobbyViewModel is now complete, and it is time to create the view!

Adding the Lobby view

The **Lobby** view simply displays a list of connected players with their avatar, gamertag and current status. To build the **LobbyView** follow these steps:

1. Right-click on the **Views** folder of the **SticksAndStone.App** project, select **Add**, and then click **New Item....**

If you are using Visual Studio 17.7 or later, click the **Show all Templates** button in the dialog that pops up; otherwise, move to the next step.

2. Under the **C# Items** node on the left, select **.NET MAUI**.
3. Select **.NET MAUI ContentPage (XAML)** and name it **LobbyView.xaml**.
4. Click **Add** to create the page.

Refer to the following screenshot to view the preceding information:

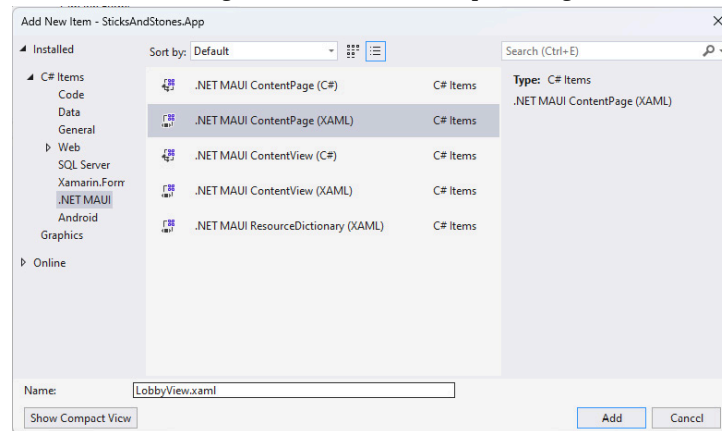


Figure 10.11 – Adding a new .NET MAUI ContentPage (XAML)

5. Open the **LobbyView.xaml.cs** file and add the following **using** declaration:

```
using SticksAndStones.ViewModels;
```

6. Make the following highlighted changes to the constructor:

```
public LobbyView(LobbyViewModel viewModel)
{
    this.BindingContext = viewModel;
    InitializeComponent();
}
```

These changes allow for dependency injection to supply the **LobbyViewModel** instance to the view, which is then assigned to **BindingContext**.

7. Open the **AppShell.xaml** file and add the following code snippet to the **ContentPage** element:

```
<ShellItem Route="Lobby">
    <ShellContent ContentTemplate="{DataTemplate views:LobbyView}" />
</ShellItem>
```

This registers the **"Lobby"** route and directs it to **LobbyView**.

8. Open the **MauiProgram.cs** file and add the following highlighted line of code:

```
builder.Services.AddTransient<Views.ConnectView>();
builder.Services.AddTransient<Views.LobbyView>();
return builder.Build();
```

This will register **LobbyView** with dependency injection so that **DataTemplate** can locate it.

9. Open the **LobbyView.xaml** file and change the **Title** attribute of the **ContentPage** element to "**Lobby**".
10. Add the following highlighted namespaces to the **LobbyView** element; they will provide us access to the classes in the **ViewModels**, **Controls**, and **Toolkit** namespaces:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:Class="SticksAndStones.Views.LobbyView">
```

11. To make IntelliSense happy with the bindings we will be adding, define the view model that the view is using by adding the **x:DataType** attribute to the **LobbyView** element, as shown:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:DataType="viewModels:LobbyViewModel"
             x:Class="SticksAndStones.Views.LobbyView">
```

12. We don't want the user to use any navigation, such as the **Shell**-provided **Back** button, other than what is provided on this page, so disable it using the highlighted code in the following listing:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:Class="SticksAndStones.Views.LobbyView"
             x:DataType="viewModels:LobbyViewModel"
             NavigationPage.HasNavigationBar="False">
```

13. Finally, set the **BackgroundColor** value of the entire view to **White**, which will make the images blend better, by adding the following highlighted code:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:Class="SticksAndStones.Views.ConnectView"
             x:DataType="viewModels:ConnectViewModel"
             Title="ConnectView"
             NavigationPage.HasNavigationBar="False"
             BackgroundColor="White">
```

14. Replace the contents of **ContentPage** with the following code snippet:

```
<RefreshView IsRefreshing="{Binding IsRefreshing}" Command="{Binding RefreshCommand}">
    <ScrollView Padding="5">
```

```

<CollectionView ItemsSource="{Binding Players}" Margin="5,5,5,0 SelectionMode="None">
</CollectionView>
</ScrollView>
</RefreshView>

```

For **LobbyView**, there is a vertical scrolling list of players. The root element is **RefreshView**. Its **IsRefreshing** attribute is bound to the **IsRefreshing** property of **LobbyViewModel**. The **Command** attribute for **RefreshView** is bound to **RefreshCommand**, which will end up executing the **RefreshInternal** method of **LobbyViewModel**. **IsRefreshing** and **RefreshCommand** are implemented in the **BaseViewModel** class. Inside **RefreshView** is **ScrollView**, which provides scrolling capability to have a large list. Inside **ScrollView** is **CollectionView**, which will display each **Player** instance as an individual item, so **ItemsSource** is bound to the **Players** property of **LobbyViewModel**. As there is no real need to select individual **Player** items, **SelectionMode** is set to **none**.

15. When the list is empty, it is nice to display something to the user so that they aren't left wondering what happened. **CollectionView** has an **EmptyView** property that is used to configure what is displayed when there are no items. Add the following code snippet immediately following the **ContentPage** start opening tag:

```

<ContentPage.Resources>
  <ContentView x:Key="BasicEmptyView">
    <StackLayout>
      <Label Text="No players available"
        Margin="10,25,10,10"
        FontAttributes="Bold"
        FontSize="18"
        HorizontalOptions="Fill"
        HorizontalTextAlignment="Center" />
    </StackLayout>
  </ContentView>
</ContentPage.Resources>

```

This defines a page resource containing **ContentView** with a **Key** value of **"BasicEmptyView"**. The view contains **StackLayout**, which has **Label** as a child with the text **"No players available"**. Appropriate styling is applied to make sure it's large enough and has enough surrounding whitespace.

16. Add the following attribute to the **CollectionView** element:

```
EmptyView="{StaticResource BasicEmptyView}"
```

This binds **BasicEmptyView** to the **EmptyView** property of **CollectionView**. *Figure 10.12* shows the result if you run the app and log in:

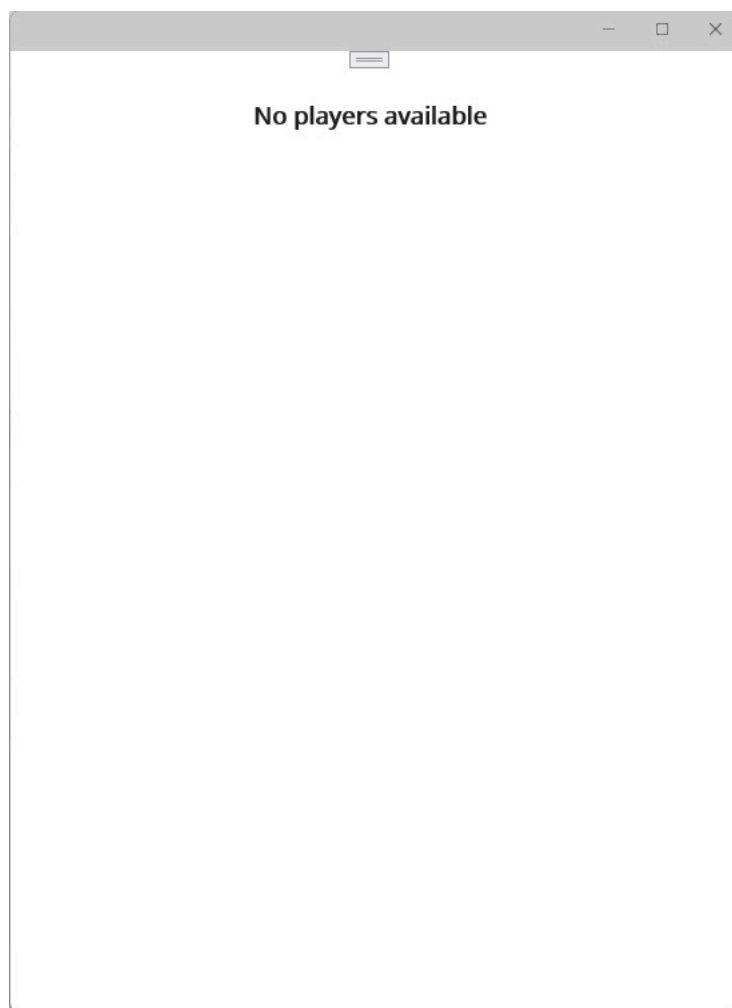


Figure 10.12 – Lobby with no players

17. The player card will also use a static resource, which just makes the file a little easier to read, and less indenting. Add the following code snippet to the **ContentView.Resources** element, under the **BasicEmptyView** element:

```
<DataTemplate x:Key="PlayerCardViewTemplate" x:DataType="viewModels:PlayerViewModel">
  <ContentView>
    <Border StrokeShape="RoundRectangle 10,10,10,10" BackgroundColor="AntiqueWhite" Padding="10">
      <Grid>
        <Grid.ColumnDefinitions>
          <ColumnDefinition Width="50" />
          <ColumnDefinition Width="4*" />
          <ColumnDefinition Width="2*" />
        </Grid.ColumnDefinitions>
        <toolkit:AvatarView Grid.Column="0" Margin="0" BackgroundColor="LightGrey" Height="50">
          <toolkit:AvatarView.ImageSource>
            <toolkit:GravatarImageSource
              Email="{Binding EmailAddress}"
              Image="MysteryPerson" />
          </toolkit:AvatarView.ImageSource>
        </toolkit:AvatarView>
        <VerticalStackLayout Grid.Column="1" Margin="10,0,0,0">
          <Label Text="{Binding GamerTag}" HorizontalTextAlignment="Start" FontSize="14">
            <Label Text="{Binding Status}" HorizontalTextAlignment="Start" FontSize="12">
          </VerticalStackLayout>
          <controls:ActivityButton Grid.Column="2" IsRunning="{Binding IsChallenging}" Text="Challenge" />
        </Grid>
      </Border>
    </ContentView>
  </DataTemplate>
```

```

        </Grid>
    </Border>
</ContentView>
</DataTemplate>

```

DataTemplate will display the player's avatar. To do so, it will use an **Image** control and **GravatarImageSource** the same way as was done in the *Creating the Connect view* section. A **DataTemplate** element is needed since this is used for **ItemTemplate**, then the obligatory **ContentView**. Then, **Border** is defined. It uses a special **Stroke** shape to round out the edges of the rectangle instead of having square corners, and a color of **AntiqueWhite** is applied as the **BackgroundColor** value. Additional shapes to use for **Border** include **Ellipse**, **Line**, **Path**, **Polygon**, **Polyline**, and **Rectangle**. See the **Border** documentation at <https://learn.microsoft.com/en-us/dotnet/maui/user-interface/controls/border> for more details. Inside **Border**, there is a **Grid** control that defines three columns. The first column contains the avatar image and has a width of **50**, the next column contains the gamer tag and the status of the player, stacked vertically, and the third column contains the **Challenge** button.

For the avatar, the **AvatarView** control from **CommunityToolkit** is used. It provides a round version of the image.

The **Challenge** button uses the **ActivityButton** control, and is bound to the **IsChallenging**, **CanChallenge**, **ChallengeStatus**, and **ChallengeCommand** properties of **PlayerViewModel**.

18. To use **PlayerCardViewTemplate** as **ItemTemplate** of **CollectionView**, add the following attribute to the **CollectionView** element:

```
ItemTemplate="{StaticResource PlayerCardViewTemplate}"
```

That completes the **Lobby** page. At this point, you should be able to launch **SticksAndStone.Functions** and connect with the **SticksAndStones.App** project to see the different layouts provided by the **Lobby** view. There is only one more page to create to complete the game, and that is the **Match** page.

Creating the Match page

The **Match** page displays the game board with the players and score. It also manages the gameplay, allowing each player to take their turn placing a stick. As each player takes their turn, the board updates to show the current state of the match. Let's get started with creating the **ViewModel** classes.

Creating the ViewModel classes

There are two different **ViewModel** classes used in the **Match** page, just as there were in the **Lobby** page, one for the game and a second for the player details.

Adding MatchPlayerViewModel

MatchPlayerViewModel is the abstraction between the **Player** model and **MatchView**. **MatchPlayerViewModel** needs to expose the **Id**, **GamerTag**, and **EmailAddress** values from the **Player** model to **MatchView**. In addition, since each player has a score, the player's score from the **Match** model is exposed to **MatchView**. There are a couple of additional properties that are needed as well:

- **IsPlayersTurn:**

This is used to determine whether **MatchPlayerViewModel** is the current player.

- **PlayerToken:**

This is used to map each player to a token to track which player has placed which stick. A token, either **-1** or **1**, is used because it makes determining a winner easier than if it was using the **Id** property, which is a **Guid**. Review the *Processing turns* section of *Chapter 9*, for a refresher on how the winner is determined.

To create **MatchPlayerViewModel**, follow these steps:

1. Create a new class named **MatchPlayerViewModel** in the **ViewModels** folder of the **SticksAndStones.App** project.
2. Modify the **using** declarations to the following at the top of the file:

```
using CommunityToolkit.Mvvm.ComponentModel;
using SticksAndStones.Models;
```

3. Add the **public** and **partial** modifiers to the class and make it inherit from **ObservableObject**, as shown:

```
public partial class MatchPlayerViewModel: ObservableObject
{
}
```

4. **MatchPlayerViewModel** is an abstraction of both the **Player** and **Match** models, which will be passed in through the constructor. Create the fields and the constructor, as shown in the following listing:

```
private readonly Player playerModel;
private readonly Match matchModel;
public MatchPlayerViewModel(Player player, Match match)
{
    this.playerModel = player;
    this.matchModel = match;
}
```

5. The **PlayerToken** property is **1** if the **Player** model is **PlayerOne** in the **Match** model; otherwise, it is **-1**. Add the **PlayerToken** property using as shown here:

```
public int PlayerToken => playerModel.Id == matchModel.PlayerOneId ? 1 : -1;
```

6. The **IsPlayersTurn** property will return **true** if the **Player** model is the **NextPlayer** of the **Match** model, as shown here:

```
public bool IsPlayersTurn => playerModel.Id == matchModel.NextPlayerId;
```

7. The **Id**, **GamerTag**, and **EmailAddress** properties all just map directly to the corresponding property from the **Player** model. This is the same implementation that was used in **PlayerViewModel** for the **Lobby** page. Use the following listing to add the properties to

MatchPlayerViewModel:

```
public Guid Id => playerModel.Id;
public string GamerTag => playerModel.GamerTag;
public string EmailAddress => playerModel.EmailAddress;
```

8. The final property that is needed for **MatchPlayerViewModel** is the **Score** property. The **Score** property is mapped to either the **PlayerOneScore** or **PlayerTwoScore** property from the **Match** model, depending on which player the **Player** model is. Use the following listing to add the **Score** property to **MatchPlayerViewModel**:

```
public int Score => playerModel.Id == matchModel.PlayerOneId ? matchModel.PlayerOneScore : matc
```

That is all there is to **MatchPlayerViewModel**. The next section will walk you through creating **MatchViewModel**.

Adding MatchViewModel

MatchViewModel needs to provide the functionality for all the gameplay. It provides the two **MatchPlayerViewModel** objects for display in the header of the page, and the board to display where sticks have been played and which stones have been captured. It also provides the needed functionality for players to take their turn and forfeit the game if they choose. To implement **MatchViewModel**, follow these steps:

1. Create a new class in the **ViewModels** folder of the **SticksAndStones.App** project named **MatchViewModel**.
2. Modify the **using** declarations section at the top of the page to match the following listing:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using CommunityToolkit.Mvvm.Messaging;
using SticksAndStones.Models;
using SticksAndStones.Services;
```

3. Add the **public** and **partial** class modifiers to the class, inherit from **ViewModelBase**, and implement **IQueryAttributable**, as shown:

```
public partial class MatchViewModel : ViewModelBase, IQueryAttributable
```

Recall that in **ConnectViewModel** and **LobbyViewModel**, when they navigate to **Match**, they pass an argument – either **MatchId** or the **Match** instance itself. **IQueryAttributable** is how that argument is passed to **MatchViewModel**. The implementation for **IQueryAttributable** is provided in a later step.

4. **MatchViewModel** only has a single dependency, **GameService**, so add a field to store the instance and a constructor to accept the instance as a parameter, as shown in the following listing:

```
private readonly GameService gameService;
public MatchViewModel(GameService gameService)
{
    this.gameService = gameService;
}
```

5. When **MatchViewModel** is loaded, it will need to process the arguments, either the **Match** instance or a **MatchId** value. Either argument will end up with a **Match** instance that is used for displaying the board in the view, and from that, create two instances of **MatchPlayerViewModel** for player one and player two. Add **match**, **playerOne**, and **playerTwo** fields to the **MatchViewModel** class to hold those instances, as shown here:

```
[ObservableProperty]
private Match match;
[ObservableProperty]
private MatchPlayerViewModel playerOne;
[ObservableProperty]
private MatchPlayerViewModel playerTwo;
```

6. **IQueryAttributable** is used to process the arguments passed to the view model. Well, it is one way of doing it. The **IQueryAttributable** interface has only one method defined, **ApplyQueryAttributes**. The .NET MAUI routing system will automatically call the **ApplyQueryAttributes** method if the view model implements the **IQueryAttributable** interface. To add the implementation for **IQueryAttributable**, use the following listing:

```
public async Task ApplyQueryAttributes(IDictionary<string, object> query)
{
    Match match = null;
    if (query.ContainsKey(Constants.ArgumentNames.Match))
    {
        match = query[Constants.ArgumentNames.Match] as Match;
    }
    if (query.ContainsKey(Constants.ArgumentNames.MatchId))
    {
        var matchId = new Guid($"{{query[Constants.ArgumentNames.MatchId]}}");
        if (matchId != Guid.Empty)
        {
            match = await gameService.GetMatchById(matchId);
        }
    }
    LoadMatch(match);
};

private void LoadMatch(Match match)
{
    if (match is null) return;
    PlayerOne = new MatchPlayerViewModel(gameService.GetPlayerById(match.PlayerOneId), match);
    PlayerTwo = new MatchPlayerViewModel(gameService.GetPlayerById(match.PlayerTwoId), match);
}
```

```

        this.Match = match;
    }

```

ApplyQueryAttributes has a single parameter *query*, which is a dictionary of key-value pairs with the key as a string and the value as an object. The key *Id* is the name of the parameter as was passed in – for example, **"Match"** or **"MatchId"**. The method will check for the existence of the **"Match"** key being present and get the value as **Match** if it is. If the **"MatchId"** key is present, then **GameService** is used to get the **Match** model from **Id**. If there is no value for *match*, then the method returns; otherwise, initialize the two instances of **GamePlayerViewModel** and store them and **Match** in the **ViewModel** properties. The **LoadMatch** method is called from **ApplyQueryAttributes** since we will need the same functionality when the **UpdateMatch** event is received.

- Before we can allow a player to choose a location to place a stick, it must be their turn. Create a property named **IsCurrentPlayersTurn** using the following listing:

```

public bool IsCurrentPlayersTurn => gameService.CurrentPlayer.Id == (Match?.NextPlayerId ?? Gui

```

- Anytime the **Match** object is updated, **IsCurrentPlayersTurn** needs to be updated as well, since it depends on values in the **Match** property. To have this happen automatically, use the **NotifyPropertyChangedFor** attribute from **CommunityToolkit**. Add the highlighted line in the following code listing:

```

[ObservableProperty]
[NotifyPropertyChangedFor(nameof(IsCurrentPlayersTurn))]
private Match match;

```

Now, whenever the **Match** property is changed, the **NotifyPropertyChanged** method will also be called for **IsCurrentPlayersTurn**. See *the Defining a ViewModel base class* section in *Chapter 2*, for a refresher on implementing the **INotifyPropertyChanged** interface.

- The game allows the player to try out different positions of the stick before committing. If this is the current player's turn, the one connected and using the app, then the **SelectStick** method will place a stick at the location chosen by the user. The choice is not sent to the server until the user clicks the **Send** button in the view; this allows the player a little time to think and try out different plays before committing. The last choice made by the player is captured in the **lastSelectedStick** field. Add the following listing to implement the **SelectStick** method:

```

int lastSelectedStick = -1;
[RelayCommand(CanExecute = nameof(IsCurrentPlayersTurn))]
private void SelectStick(string arg)
{
    if (gameService.CurrentPlayer is null) return;
    if (Match is null) return;

    if (int.TryParse(arg, out var pos))

```

```

    {
        pos--; // adjust for 0 based indexes
        if (lastSelectedStick != -1 && lastSelectedStick != pos)
            Match.Sticks[lastSelectedStick] = 0;
        if (Match.Sticks[pos] != 0)
            return;
        Match.Sticks[pos] = gameService.CurrentPlayer.Id == PlayerOne.Id ? PlayerOne.PlayerToke
        lastSelectedStick = pos;
        OnPropertyChanged(nameof(Match));
    }
}

```

The value of **-1** for **lastSelectedStick** is used to mean no stick. The **SelectStick** method is exposed as a **Command** instance via **RelayCommandAttribute**. The **Is CurrentPlayersTurn** property is used to determine whether the command can execute. Recall from *Chapter 9* that the **Sticks** elements will have one of three values: **-1** for player one, **0** for empty, and **1** for player two. After determining whether the stick position is valid, the method raises the **OnPropertyChanged** event for the **Match** property, which causes the bindings to update.

10. After deliberating on which position to place their next stick, the player has three options: send their move to the server and end their turn, be indecisive and undo their move, or give up and exit the match. Add the **Play** method to **MatchViewModel** using the following code snippet:

```

[RelayCommand]
private async Task Play()
{
    if (lastSelectedStick == -1)
    {
        await Shell.Current.CurrentPage.DisplayAlert("Make a move", "You must make a move before", "Ok");
        return;
    }
    if (await Shell.Current.CurrentPage.DisplayAlert("Make a move", "Are you sure this is the move you want to make?", "Yes", "No"))
    {
        var (newMatch, error) = await gameService.EndTurn(Match.Id, lastSelectedStick);
        if (error is not null)
        {
            await Shell.Current.CurrentPage.DisplayAlert("Error in move", error, "Ok");
            return;
        }
        lastSelectedStick = -1;
    }
}

```

The **Play** method is exposed as a **Command** so that it can be bound to by UI elements.

11. The **Undo** method is called when the player taps the **Undo** button. This method resets the value of the **lastSelectedStick** position and the value of **lastSelectedStick**. Add the **Undo** method, as shown in the following code listing:

```

[RelayCommand]
private async Task Undo()
{
    if (lastSelectedStick != -1)
    {
        lastSelectedStick = -1;
    }
}

```

```

    {
        if (await Shell.Current.CurrentPage.DisplayAlert("Undo your move", "Are you sure you do
        {
            OnPropertyChanging(nameof(Match));
            Match.Sticks[lastSelectedStick] = 0;
            OnPropertyChanged(nameof(Match));
            lastSelectedStick = -1;
            return;
        }
    }
}

```

Again, the **RelayCommand** attribute is applied to the method to allow it to be bound to by UI elements.

The **Forfeit** method is called when the player uses the **Back** button to go back to the Lobby. If the match is still in progress, then the player is warned that leaving the match forfeits to the opposing player. Use the following code listing to add the **Forfeit** method to the

MatchViewModel class:

```

[RelayCommand]
private async Task Forfeit()
{
    var returnToLobby = true;
    if (!Match.Completed)
    {
        returnToLobby = await Shell.Current.CurrentPage.DisplayAlert("W A I T", "Returning to t
    if (returnToLobby)
    {
        await Shell.Current.GoToAsync("///Lobby");
    }
}

```

12. When the opposing player sends their move to the server, it is received in the app as a **MatchUpdated** event from the SignalR service. Add the handler for the **MatchUpdated** event using the following listing:

```

void OnMatchUpdated(object r, Messages.MatchUpdated m)
{
    LoadMatch(m.Value);
    if (Match.WinnerId != Guid.Empty && Match.Completed == true)
    {
        MainThread.InvokeOnMainThreadAsync(async () =>
        {
            if (Match.WinnerId == gameService.CurrentPlayer.Id)
            {
                await Shell.Current.CurrentPage.DisplayAlert("Congratulations!", $"You are vict
            }
            else
            {
                await Shell.Current.CurrentPage.DisplayAlert("Bummer!", $"You were defeated, be
            }
        });
        return;
    }
}

```

13. To register the **MatchUpdated** event handler, the **Register** method is called from **OnActivated**, and **UnRegister** is called from

OnDeactivated, as shown in the following listing:

```
protected override void OnActivated()
{
    Messenger.Register(this, (MessageHandler<object, Messages.MatchUpdated>)OnMatchUpdated);
}
protected override void OnDeactivated()
{
    Messenger.Unregister<Messages.MatchUpdated>(this);
}
```

14. Register **MatchViewModel** with dependency injection by adding the following highlighted line of code to the **CreateMauiApp** method in the **MauiProgram.cs** file:

```
builder.Services.AddTransient<ViewModels.ConnectViewModel>();
builder.Services.AddTransient<ViewModels.LobbyViewModel>();
builder.Services.AddTransient<ViewModels.MatchViewModel>();
builder.Services.AddTransient<Views.ConnectView>();
builder.Services.AddTransient<Views.LobbyView>();
```

WHY IS IT CALLED IQUERYATTRIBUTABLE? THAT FEELS AWKWARD

*The reason behind the name of the interface is that naming things is hard. The system for passing arguments to a view model can be either declarative or not. The declarative way uses **QueryPropertyAttribute** to map the query parameter to a property on the view model. If you choose not to use the attributes, but instead handle the mapping yourself manually, you declare your class **IQueryAttributable**, as in I could have used **QueryPropertyAttribute** but I choose not to. For more information, visit <https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/shell/navigation#pass-data>.*

Adding the Match view

This page is complex, so we are going to break it down into smaller, more manageable chunks. First, the basic page layout is defined including the commands that are available to the player: **Play**, **Undo**, and **Forfeit**. Next, the scoreboard area is defined with the player's gamer tag, Gravatar, and scores. Finally, the game board is defined and laid out in a three-by-three grid. Let's get started by creating the view and the layout.

Create the view

The **Match** view is not unlike any of the other views that have been created except that it has many more elements than preview views. Let's get started by creating the view and some basic elements by following these steps:

1. Right-click on the **Views** folder of the **SticksAndStone.App** project, select **Add**, and then click **New Item...**
If you are using Visual Studio 17.7 or later, click the **Show all Templates** button in the dialog that pops up; otherwise, move to the

next step.

2. Under the **C# Items** node on the left, select **.NET MAUI**.
3. Select **.NET MAUI ContentPage (XAML)** and name it **MatchView**.
4. Click **Add** to create the page.

Refer to the following screenshot to view the preceding information:

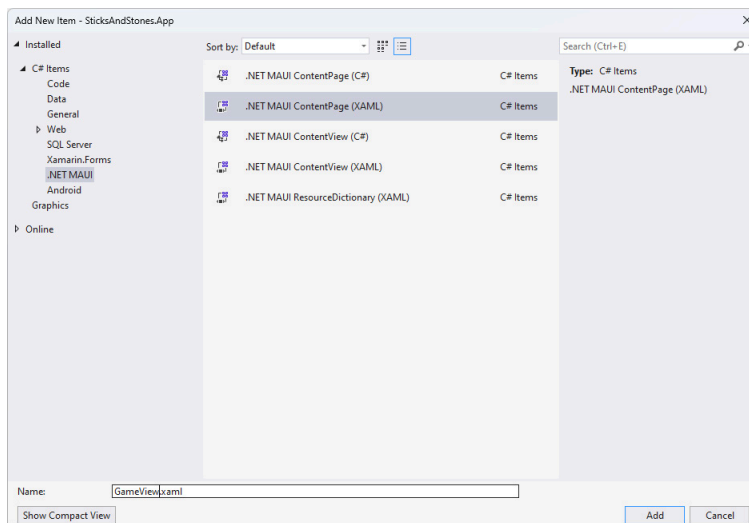


Figure 10.13 – Adding a new .NET MAUI ContentPage (XAML)

5. Open the **MatchView.xaml.cs** file and add the following **using** declaration:

```
using SticksAndStones.ViewModels;
```

6. Make the following highlighted changes to the constructor:

```
public MatchView(MatchViewModel viewModel)
{
    this.BindingContext = viewModel;
    InitializeComponent();
}
```

These changes allow for dependency injection to supply the **MatchViewModel** instance to the view, which is then assigned to **BindingContext**.

7. Open the **AppShell.xaml** file and add the following code snippet to the **ContentPage** element:

```
<ShellItem Route="Match">
    <ShellContent ContentTemplate="{DataTemplate views:MatchView}" />
</ShellItem>
```

This registers the **"Match"** route and directs it to **MatchView**.

8. Open the **MauiProgram.cs** file and add the following highlighted line of code:

```
builder.Services.AddTransient<Views.ConnectView>();
builder.Services.AddTransient<Views.LobbyView>();
builder.Services.AddTransient<Views.MatchView>();
return builder.Build();
```


This will register **MatchView** with dependency injection so that

DataTemplate can locate it.

9. Open the **MatchView.xaml** file and remove the **Title** attribute of the **ContentPage** element.
10. Add the following highlighted namespaces to the **MatchView** element. They will provide us access to the classes in the **ViewModels**, **Converters**, and **Controls** namespaces:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:Class="SticksAndStones.Views.GameView">
```

11. To make IntelliSense happy with the bindings we will be adding, define the view model that the view is using by adding the **x:DataType** attribute to the **MatchView** element, as shown:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:viewModels="clr-namespace:SticksAndStones.ViewModels"
             xmlns:controls="clr-namespace:SticksAndStones.Controls"
             xmlns:toolkit="http://schemas.microsoft.com/dotnet/2022/maui/toolkit"
             x:DataType="viewModels:GameViewModel"
             x:Class="SticksAndStones.Views.GameView">
```

MatchView uses a few icons from the Font Awesome font library, so we will need to download and install the library so that it is available in the app.

Downloading and configuring Font Awesome

Font Awesome is a free collection of images packaged into a font. .NET MAUI has excellent support for using Font Awesome in toolbars, navigation bars, and all over the place. It's not strictly needed to make this app, but we think that it's worth the extra round trip since you are most likely going to need something like this in your new killer app.

Downloading the font is straightforward. Please note the renaming of the file – it is not really needed but it's easier to edit configuration files and such if they have a simpler name. Follow these steps to acquire and copy the font to each project:

1. Browse to <https://fontawesome.com/download>.
2. Click the **Free for Desktop** button to download Font Awesome.
3. Unzip the downloaded file, then locate the **otfs** folder.
4. Rename the **Font Awesome 5 Free-Solid-900.otf** file to **FontAwesome.otf** (you can keep the original name, but it's just less to type if you rename it). Your filename may be different since Font Awesome is continually updating but it should be similar.
5. Copy **FontAwesome.otf** to the **Resources/Fonts** folder in the **SticksAndStones.App** project.

It would be nice if all that was needed was to copy the font file into the project folders. A lot does happen with just that action. The default .NET MAUI template includes all the fonts in the **Resources/Fonts** folder with the following item definition in the **News.csproj** file:

```
<!-- Custom Fonts -->
<MauiFont Include="Resources\Fonts\*" />
```

This ensures that the font files are processed and included in the app package automatically. What is left is to register the font with the .NET MAUI runtime so it is available to our XAML resources. To do that, add the following highlighted line to the **MauiProgram.cs** file:

```
.ConfigureFonts(fonts =>
{
    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
    fonts.AddFont("FontAwesome.otf", "FontAwesome");
})
```

This line adds an alias that we can use in the next section to create static resources. The first parameter is the filename for the font file, and the second is the alias for the font that you can use in the **FontFamily** attribute.

Defining the layout

Now that Font Awesome is installed and configured in .NET MAUI, **TitleView** can use it. Add the custom title area and the main layout by following these steps:

1. First, override **TitleView** of the **Shell** element and provide a new container to hold the buttons:

```
<Shell.TitleView>
    <Grid>
        <HorizontalStackLayout HorizontalOptions="Start">
        </HorizontalStackLayout>
        <HorizontalStackLayout HorizontalOptions="End">
        </HorizontalStackLayout>
    </Grid>
</Shell.TitleView>
```

The buttons are arranged in two segments, one aligned to the left or start of the window and the other aligned to the right or end of the window.

2. The player can at any point decide they no longer wish to continue playing. To exit the match, the player can use the **Back** button to return to the Lobby. When the button is tapped, the player is prompted to ensure that they no longer wish to continue playing. To add the **Back** button to the **Start** section of **TitleView** and bind **ForfeitCommand** in **MatchViewModel**, add the highlighted code from the following snippet :

```
<HorizontalStackLayout HorizontalOptions="Start">
  <ImageButton Command="{Binding ForfeitCommand}" ToolTipProperties.Text="Return to the lobby"
    <ImageButton.Source>
      <FontImageSource Glyph="&#xf0a8;" FontFamily="FontAwesome" Color="White" Size="28"
    </ImageButton.Source>
  </ImageButton>
</HorizontalStackLayout>
```

3. When it is the player's turn, they have two buttons that are enabled, **Play** and **Undo**. The **Play** and **Undo** buttons are placed in the **TitleView** area of the .NET MAUI page. Add the following highlighted code to add the **Play** and **Undo** buttons to **TitleView**:

```
<HorizontalStackLayout HorizontalOptions="End">
  <ImageButton Command="{Binding UndoCommand}" IsVisible="{Binding IsCurrentPlayersTurn}" Too
    <ImageButton.Source>
      <FontImageSource Glyph="&#xf0e2;" FontFamily="FontAwesome" Color="White" Size="28"
    </ImageButton.Source>
  </ImageButton>
  <ImageButton Command="{Binding PlayCommand}" IsVisible="{Binding IsCurrentPlayersTurn}" Too
    <ImageButton.Source>
      <FontImageSource Glyph="&#xf1d8;" FontFamily="FontAwesome" Color="White" Size="28"
    </ImageButton.Source>
  </ImageButton>
</HorizontalStackLayout>
```

4. Remove the default **VerticalStackLayout** element in **ContentView** and add the following code:

```
<ContentView>
  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="4*" />
      <RowDefinition Height="2*" />
      <RowDefinition Height="6*" />
      <RowDefinition Height="2*" />
    </Grid.RowDefinitions>
  </Grid>
</ContentView>
```

This adds a **Grid** control with four rows. The first and third rows will contain the scoreboard and the game board, respectively, while the second and fourth rows are padding.

The main layout is ready. Next, the scoreboard is added to the first row of the layout.

Creating the scoreboard

The scoreboard contains each player's avatar, gamer tag, and score. The elements are bound to the respective fields of **MatchPlayerViewModel** for the player. The **Match** object has two properties, **PlayerOne** and **PlayerTwo**, each of which is a **MatchPlayerViewModel**. To add the scoreboard, follow these steps:

1. Each player is identified by a different color. To add each color as a resource, open the **Colors.xaml** file in the **Resources/Styles** folder of the **SticksAndStones.App** project and add the following lines to the **ResourceDictionary** element:

```
<Color x:Key="PlayerOne">#6495ED</Color>
<Color x:Key="PlayerTwo">#CD5C5C</Color>
```

2. The scoreboard uses **HorizontalStackLayout** for the outer container. Add the following code to the **Grid** element:

```
<HorizontalStackLayout Grid.Row="0" HorizontalOptions="CenterAndExpand" Margin="10" BindableLayout
</HorizontalStackLayout>
```

HorizontalStackLayout is assigned to row **0** of the **Grid**, and its contents are bound to the **Players** property of the view model, using **BindableLayout.ItemsSource**. **BindableLayout** is the underlying interface that supports all layout controls, such as **AbsoluteLayout** and **FlexLayout**.

3. Each player will have their own card within **HorizontalStackLayout**. Since the control is bound to the **Players** property, which is an array of **MatchPlayerViewModels**, the **BindableLayout.ItemTemplate** property provides the view that each item in **Players** is displayed with. The cards are laid out using a **Border** element and nested **VerticalStackLayout** elements. Add the following highlighted code to **HorizontalStackLayout**:

```
<HorizontalStackLayout Grid.Row="0" HorizontalOptions="CenterAndExpand" Margin="10">
  <BindableLayout.ItemTemplate>
    <DataTemplate>
      <Border x:DataType="viewModels:MatchPlayerViewModel" Padding="0" Margin="2" StrokeS
        <VerticalStackLayout Padding="2" HorizontalOptions="Center">
          </VerticalStackLayout>
        </Border>
      </DataTemplate>
    </BindableLayout.ItemTemplate>
  </HorizontalStackLayout>
```

4. The **Border** element is the outermost container for the player card. To set the border color and background color of the **Border** element based on **PlayerToken**, triggers are used (<https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/triggers>) – specifically, **DataTrigger** is used to set attribute values based on some other value. Add the following code to the **Border** element:

```
<Border.Triggers>
  <DataTrigger TargetType="Border" Binding="{Binding PlayerToken}" Value="1" >
    <Setter Property="Stroke" Value="{StaticResource PlayerOne}" />
    <Setter Property="BackgroundColor" Value="{StaticResource PlayerOne}" />
  </DataTrigger>
  <DataTrigger TargetType="Border" Binding="{Binding PlayerToken}" Value="-1" >
    <Setter Property="Stroke" Value="{StaticResource PlayerTwo}" />
    <Setter Property="BackgroundColor" Value="{StaticResource PlayerTwo}" />
</Border.Triggers>
```

```
</DataTrigger>
</Border.Triggers>
```

The **DataTrigger** binding attribute is compared to the **Value** attribute. If they are equal, then the **Setter** elements of **DataTrigger** are executed. In this case, if the **PlayerToken** property is **-1**, then set the **Stroke** and **BackgroundColor** attributes of **Border** to the **PlayerOne** color that was defined in *step 1*. Otherwise, if the **PlayerToken** property is equal to **-1**, then set the **Stroke** and **BackgroundColor** attributes to the **PlayerTwo** color.

5. **VerticalStackLayout** contains another **VerticalStackLayout** and **Border** element, as shown in the following highlighted code:

```
<VerticalStackLayout BackgroundColor="{Binding PlayerToken, Converter={StaticResource PlayerToC
  <VerticalStackLayout>
  </VerticalStackLayout>
  <Border Padding="0" WidthRequest="96" StrokeShape="RoundRectangle 10,10,10,10" StrokeThickn
    <Image IsVisible="{Binding IsPlayersTurn}" Source="hstick.jpeg" Aspect="AspectFit" Maxi
    </Border>
  </VerticalStackLayout>
```

VerticalStackLayout will be used to hold **GamerTag**, **AvatarImage**, and the player's score, which is added in the next step. **Border** contains a horizontal stick image whose **IsVisible** attribute is bound to the **IsPlayersTurn** property. The stick is used as a visual indicator of which player's turn it is. If it is not the player's turn, the image is not displayed.

6. Within the second **VerticalStackLayout** are a **Label** and a **FlexLayout**. Add the following highlighted code:

```
<VerticalStackLayout>
  <Label Text="{Binding GamerTag}" HorizontalOptions="FillAndExpand" HorizontalTextAlignment=
  <FlexLayout Margin="3">
  </FlexLayout>
</VerticalStackLayout>
```

7. **FlexLayout** contains the visual elements to display **AvatarImage** and **Score**. Add the following highlighted code to **FlexLayout**:

```
<FlexLayout Margin="3">
  <toolkit:AvatarView FlexLayout.Order="0" Margin="0" BackgroundColor="LightGrey" HeightReque
    <toolkit:AvatarView.ImageSource>
      <toolkit:GravatarImageSource
        Email="{Binding EmailAddress}"
        Image="MysteryPerson" />
    </toolkit:AvatarView.ImageSource>
    <toolkit:AvatarView.Triggers>
      <DataTrigger TargetType="toolkit:AvatarView" Binding="{Binding Path=PlayerToken}" V
        <Setter Property="FlexLayout.Order" Value="1" />
      </DataTrigger>
    </toolkit:AvatarView.Triggers>
  </toolkit:AvatarView>
  <Label FlexLayout.Order="1" Text="{Binding Score}" FontSize="48" Padding="5" MinimumWidthRe
    <Label.Triggers>
      <DataTrigger TargetType="Label" Binding="{Binding Path=PlayerToken}" Value="-1">
        <Setter Property="FlexLayout.Order" Value="0" />
      </DataTrigger>
```

```

</Label.Triggers>
</Label>
</FlexLayout>

```

By using the **FlexLayout** control, the order in which the **FlexLayout** children are displayed is governed by the **FlexLayout.Order** attribute. Similar to **Grid** with its **Grid.Row** and **Grid.Column** properties on its children, the **Order** attribute is set on the child. The order of the children in **FlexLayout** is changed through the use of **Trigger**. On **AvatarView**, **DataTrigger** will set the **FlexLayout.Order** attribute to "1" if the **PlayerToken** property is equal to -1, which is **PlayerTwo**. On the **Label**, **DataTrigger** sets the **FlexLayout.Order** attribute to "0", effectively swapping the two elements.

And that concludes the scoreboard. The final part of **MatchView** is the largest: the board. Read on to learn how to create the board visuals.

Creating the game board

The game board is composed of three different elements. These elements are dots in the corners of each square, sticks (both horizontal and vertical), and stones. These elements are laid out as shown here:



Figure 10.14 – The game board

The board uses a **Grid** control to provide the basic layout. Using 7 columns and 7 rows will provide cells for each of the elements: 16 dots, 9 stones, and 24 sticks. Add the following code to provide the basic layout of the game board to the top-level **Grid** element:

```

<Grid Grid.Row="2" BackgroundColor="White" Margin="10,40,10,0" MaximumHeightRequest="410" MaximumW
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="1*" />
    <ColumnDefinition Width="5*" />
    <ColumnDefinition Width="1*" />

```

```

        <ColumnDefinition Width="5*" />
        <ColumnDefinition Width="1*" />
        <ColumnDefinition Width="5*" />
        <ColumnDefinition Width="1*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="1*" />
        <RowDefinition Height="4*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="4*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="4*" />
        <RowDefinition Height="1*" />
    </Grid.RowDefinitions>
</Grid>

```

Let's start by adding the corners to the grid, since they are the simplest. To define a corner, use a **Label** with the text "⚫", which is the hexadecimal character code for a dot. To center the dot horizontally and vertically, set **HorizontalOptions** and **VerticalOptions** to "Center". Your basic element would look like the following:

```
<Label Text="&#x26AB" HorizontalOptions="Center" VerticalOptions="Center" />
```

Without the **Grid.Row** and **Grid.Column** attributes, the **Label** will be put into row 0 and column 0. There are 16 corners in the grid, and they occupy all the even-numbered cells, so (0,0), (0,2), (0,4), (0,6), (2,0), (2,2), and so on. Fully defined labels for the first row would look like the following:

```

<Label Grid.Row="0" Grid.Column="0" Text="&#x26AB" HorizontalOptions="Center" VerticalOptions="Center" />
<Label Grid.Row="0" Grid.Column="2" Text="&#x26AB" HorizontalOptions="Center" VerticalOptions="Center" />
<Label Grid.Row="0" Grid.Column="4" Text="&#x26AB" HorizontalOptions="Center" VerticalOptions="Center" />
<Label Grid.Row="0" Grid.Column="6" Text="&#x26AB" HorizontalOptions="Center" VerticalOptions="Center" />

```

When you work this out for all 16 rows, that's a lot of duplication of the **Text**, **HorizontalOptions**, and **VerticalOptions** attributes. By using a **Style** element, that duplication can be eliminated. A **Style** element contains **Setter** elements such as the **DataTrigger** elements. When **Style** is applied to the element, the **Setter** elements are used to update the target element's attributes. Use the following steps to add the corner elements to the **Grid** control using **Style**:

1. Add the following **Style** element to the **ContentPage.Resources** element:

```

<Style x:Key="dotLabel"
    TargetType="Label">
    <Setter Property="Text" Value="&#x26AB;" />
    <Setter Property="HorizontalOptions" Value="Center" />
    <Setter Property="VerticalOptions" Value="Center" />
</Style>

```

This **Style** element is identified by the **x:Key** attribute.

2. Add a **Label** to the **Grid** control created at the start of this section.

3. Set the **Grid.Row** attribute of the **Label** to **0**.
4. Set the **Grid.Column** attribute of the **Label** to **0**.
5. Set the **Style** attribute to the **{StaticResource dotLabel}** value. The **Style** attribute is used to specify which style should be applied to the element. Since **Style** is defined in the **ContentView.Resources** element, it is a **StaticResource**.
6. The completed **Label** should look like the following:

```
<Label Grid.Row="0" Grid.Column="0" Style="{StaticResource dotLabel}" />
```

7. Now, copy the **Label** just created and increase the **Grid.Column** value by two, and repeat this step until you have four **Label** elements with the same **Grid.Row** value.
8. Copy the last **Label** created in *step 7* and increase the **Grid.Row** value by two and reset the value of **Grid.Column** to **0**. Now, repeat *step 7* using the updated **Grid.Row** value, and stop when there are four labels with a **Grid.Row** value of **6**.
9. The labels should look like the following listing:

```
<Label Grid.Row="0" Grid.Column="0" Style="{StaticResource dotLabel}" />
<Label Grid.Row="0" Grid.Column="2" Style="{StaticResource dotLabel}" />
<Label Grid.Row="0" Grid.Column="4" Style="{StaticResource dotLabel}" />
<Label Grid.Row="0" Grid.Column="6" Style="{StaticResource dotLabel}" />
<Label Grid.Row="2" Grid.Column="0" Style="{StaticResource dotLabel}" />
<Label Grid.Row="2" Grid.Column="2" Style="{StaticResource dotLabel}" />
<Label Grid.Row="2" Grid.Column="4" Style="{StaticResource dotLabel}" />
<Label Grid.Row="2" Grid.Column="6" Style="{StaticResource dotLabel}" />
<Label Grid.Row="4" Grid.Column="0" Style="{StaticResource dotLabel}" />
<Label Grid.Row="4" Grid.Column="2" Style="{StaticResource dotLabel}" />
<Label Grid.Row="4" Grid.Column="4" Style="{StaticResource dotLabel}" />
<Label Grid.Row="4" Grid.Column="6" Style="{StaticResource dotLabel}" />
<Label Grid.Row="6" Grid.Column="0" Style="{StaticResource dotLabel}" />
<Label Grid.Row="6" Grid.Column="2" Style="{StaticResource dotLabel}" />
<Label Grid.Row="6" Grid.Column="4" Style="{StaticResource dotLabel}" />
<Label Grid.Row="6" Grid.Column="6" Style="{StaticResource dotLabel}" />
```

Now that the corners are done, we can start on the game pieces: the sticks and stones. Since the sticks and stones have some similarities, we can create a common control to help display them all. However, they are visualized entirely differently. What is needed is a common interface to define the **BindableProperty** properties and use that on different layouts. **.NET MAUI** uses **ControlTemplate** resources to allow for the customization, or even complete replacement, of the visual elements that comprise a control. Many controls in **.NET MAUI** can be customized using a **ControlTemplate**, if they derive from **ContentView** or **ContentPage**. Let's get started with the sticks and stones by adding the custom control, then the **ControlTemplate** resources for sticks and stones, by following these steps:

1. Create a new class in the **Controls** folder of the **SticksAndStones.App** project named **GamePieceView**.
2. Update the class definition to match the following listing:


```
namespace SticksAndStones.Controls;
public partial class GamePieceView : ContentView
{
}
```

3. Add a **string** property and a **BindableProperty** property named **GamePiecePosition** and **GamePiecePositionProperty**, respectively, as shown in the following listing:

```
public static readonly BindableProperty GamePiecePositionProperty = BindableProperty.Create(
    name: nameof(GamePiecePosition),
    type: typeof(string),
    defaultValue: null,
    propertyFlags: BindableProperty.DefaultPropertyFlags | BindableProperty.DefaultPropertyFlags.ReadOnly | BindableProperty.DefaultPropertyFlags.NonBindable
);

public string GamePiecePosition
{
    get => (string)GetValue(GamePiecePositionProperty);
    set => SetValue(GamePiecePositionProperty, value);
}
```

GamePiecePosition is used to determine the array index in either **Sticks** or **Stones** properties on **GameViewModel**.

4. Add an **int** property and **BindableProperty** named **GamePieceState** and **GamePieceStateProperty**, respectively, as shown in the following listing:

```
public static readonly BindableProperty GamePieceStateProperty = BindableProperty.Create(
    name: nameof(GamePieceState),
    type: typeof(int),
    defaultValue: 0,
    propertyFlags: BindableProperty.DefaultPropertyFlags | BindableProperty.DefaultPropertyFlags.ReadOnly | BindableProperty.DefaultPropertyFlags.NonBindable
);

public int GamePieceState
{
    get => (int)GetValue(GamePieceStateProperty);
    set => SetValue(GamePieceStateProperty, value);
}
```

GamePieceState is the owner of the piece: **1** for **PlayerOne**, **0** for no one, and **-1** for **PlayerTwo**.

5. Add a **string** property and **BindableProperty** named **GamePieceDirection** and **GamePieceDirectionProperty**, respectively, as shown in the following listing:

```
public static readonly BindableProperty GamePieceDirectionProperty = BindableProperty.Create(
    name: nameof(GamePieceDirection),
    type: typeof(string),
    defaultValue: null,
    propertyFlags: BindableProperty.DefaultPropertyFlags | BindableProperty.DefaultPropertyFlags.ReadOnly | BindableProperty.DefaultPropertyFlags.NonBindable
);

public string GamePieceDirection
{
    get => (string)GetValue(GamePieceDirectionProperty);
    set => SetValue(GamePieceDirectionProperty, value);
}
```

GamePieceDirection is only needed for **Sticks** and is either **Horizontal** or **Vertical**.

6. Open the **MatchView.Xaml** file again and let's add a control template for all the sticks. Add the following snippet to the **ContentView.Resources** element:

```
<ControlTemplate x:Key="StickViewControlTemplate">
</ControlTemplate>
```

This defines a **ControlTemplate** element with a key of **StickViewControlTemplate**. The key is used to apply the **ControlTemplate** element to the control.

7. There are two elements to each stick visual: the number displayed on a label, and the stick image, which uses the **Image** control inside a border to give it the outline, colored by the player that placed the stick. The other interesting aspect is that the **Label** and **Border** controls need to be layered on top of one another. To accomplish this, a **Grid** control is used and both elements are placed in the same cell. To add the **Grid**, **Label**, **Border**, and **Image** controls, use the following listing, and add them to the **ControlTemplate** element:

```
<Grid Margin="0" Padding="0">
  <Label Text="{TemplateBinding GamePiecePosition}" IsVisible="False" HorizontalTextAlignment
</Label>
  <Border Padding="3" BackgroundColor="Transparent" StrokeShape="RoundRectangle 5" Stroke="Tr
    <Image Aspect="Fill">
    </Image>
  </Border>
</Grid>
```

Grid has **Margin** and **Padding** values of **0** so that it doesn't occupy any screen real estate. The **Label** control's **Text** attribute is bound using **TemplateBinding** to the **GamePiecePosition** property. **TemplateBinding** differs slightly from **Binding** in that **TemplateBinding** uses the control this **ControlTemplate** is applied to as **DataContext**. Since this **ControlTemplate** will be applied to instances of **GamePieceView**, it will bind to the **Bindable** properties of those controls.

Inspecting the **Image** control from *step 7*, you'll find that it doesn't specify which image is displayed. For **Sticks**, one of two images is displayed: either **hstick.jpeg** for horizontal sticks or **vstick.jpeg** for vertical sticks, and if there is no stick at that location, then the control should not be visible. The following listing uses **DataTrigger** to set the values of **IsVisible** and **Source** of the **Image** control using **TemplateBinding** to the **GamePieceState** and **GamePieceDirection** properties. Add this code to the **Image** control of **ControlTemplate**:

```
<Image.Triggers>
  <DataTrigger TargetType="Image" Binding="{TemplateBinding Path=GamePieceState}" Value="0">
    <Setter Property="IsVisible" Value="False" />
  </DataTrigger>
  <DataTrigger TargetType="Image" Binding="{TemplateBinding Path=GamePieceDirection}" Value="
    <Setter Property="Source" Value="hstick.jpeg" />
  </DataTrigger>
  <DataTrigger TargetType="Image" Binding="{TemplateBinding Path=GamePieceDirection}" Value="
    <Setter Property="Source" Value="vstick.jpeg" />
  </DataTrigger>
</Image.Triggers>
```

8. The **Border** control also uses **DataTrigger** to outline the stick in the color of the player that placed the stick. Add the following code to the **Border** element, after **Image**:

```
<Border.Triggers>
  <DataTrigger TargetType="Border" Binding="{TemplateBinding GamePieceState}" Value="1" >
    <Setter Property="Stroke" Value="{StaticResource PlayerOne}" />
  </DataTrigger>
  <DataTrigger TargetType="Border" Binding="{TemplateBinding GamePieceState}" Value="-1" >
```

```
<Setter Property="Stroke" Value="{StaticResource PlayerTwo}" />
</DataTrigger>
</Border.Triggers>
```

Two triggers are needed to switch between **PlayerOne** (1) and **PlayerTwo** (-1). The **Stroke** attribute of the **Border** control is set to the color resource of the player. If neither trigger is active, then the default **Stroke** value of **Transparent** from the **Border** element is used. In this way, if there is no stick, **GamePieceState** is 0, and the border is transparent. If **GamePieceState** is 1, then **Stroke** will have the color defined by the resource named **PlayerOne**, and if **GamePieceState** is -1, then the **Stroke** value will be the resource named **PlayerTwo**.

9. When the user is making their move during their turn, they will tap or click on the label to place their stick in that position. To call **SelectStickCommand** when that occurs, the **Border** control binds **TapGestureRecognizer** to the **GameViewModel.SelectStickCommand** property and passes **GamePiecePosition** along as a parameter. Add the following listing to the **Border** element, after the **Border.Triggers** element:

```
<Border.GestureRecognizers>
  <TapGestureRecognizer Command="{Binding Source={RelativeSource AncestorType={x:Type viewMod
</Border.GestureRecognizers>
```

10. Finally, take a close look at the **Label** element; you will see that the **IsVisible** attribute is set to **False**. If there is no stick placed at this position, then we need the label with the position displayed. That can be accomplished by using **DataTrigger**; the label's **IsVisible** property can be set to **True**, making the label visible if **GamePieceState** is 0, meaning no stick has been placed there yet. Add the following listing to the **Label** element:

```
<Label.Triggers>
  <DataTrigger TargetType="Label" Binding="{TemplateBinding Path=GamePieceState}" Value="0">
    <Setter Property="IsVisible" Value="True" />
  </DataTrigger>
</Label.Triggers>
```

That completes the control template for the sticks. Next, create the control template for **Stones** by following these steps:

1. Right below the **ControlTemplate** created for the sticks, add the following code:

```
<ControlTemplate x:Key="StoneViewControlTemplate">
</ControlTemplate>
```

Just like with the control template for sticks, the **ControlTemplate** uses a key to locate the right template.

2. The **Stones** template is a little less complex than the **Sticks** template. Here, we just have the **Border** control with an **Image** control as a child. **DataTrigger** is used again to select the right border color, and if the stones are not present, then the border is not visible. Use the following code sample and add it to the **ControlTemplate** created in *step 1*:

```

<Border Margin="3" Padding="5" HorizontalOptions="Center" VerticalOptions="Center" StrokeShape=
  <Border.Triggers>
    <DataTrigger TargetType="Border" Binding="{TemplateBinding GamePieceState}" Value="0">
      <Setter Property="IsVisible" Value="False" />
    </DataTrigger>
    <DataTrigger TargetType="Border" Binding="{TemplateBinding GamePieceState}" Value="1" >
      <Setter Property="Stroke" Value="{StaticResource PlayerOne}" />
    </DataTrigger>
    <DataTrigger TargetType="Border" Binding="{TemplateBinding GamePieceState}" Value="-1"
      <Setter Property="Stroke" Value="{StaticResource PlayerTwo}" />
    </DataTrigger>
  </Border.Triggers>
  <Image Source="stones.jpeg" Aspect="Fill" />
</Border>

```

You may have noticed a difference between the triggers in this listing versus the triggers in the **Sticks** control template. In **Sticks**, the **IsVisible** attribute was set on **Image**, not **Border**, and you may want to know why that is. The explanation is simple; if the border is not visible, it will not receive **TapGesture** events. The **Grid** element cannot register **GestureRecognizer**, so the event cannot be captured there either.

The **ControlTemplates** that are needed for the stick and stone images are in place; now, they need to be associated with the **GamePieceView** control elements. A **Style** can set the **ControlTemplate** property of a **GamePieceView** element, but how will it determine that this element is a stick or a stone? **Style** elements have a **Class** property that can be used to further refine which styles are applied to a control. If the control has a matching class name listed in its **StyleClass** attribute, then that **Style** element is applied. Let's use sticks as an example, by following these steps:

1. Add a new **Style** element to the **ContentView.Resources** element, as shown in the following listing:

```

<Style TargetType="controls:GamePieceView"
  Class="Stick">
  <Setter Property="ControlTemplate"
    Value="{StaticResource StickViewControlTemplate}" />
</Style>

```

This style is only applied to elements that are of the **GamePiece** type and have the **Stick** class listed in the **StyleClass** attribute. A matching element might look like the following:

```

<controls:GamePieceView Grid.Row="0" Grid.Column="1" StyleClass="Stick"
  GamePiecePosition="01" GamePieceState="{Binding Game.Sticks[0]}" GamePi

```

The highlighted sections show the parts of the control that are used to match the **Style** element. **StyleClass** can have more than one name listed; just use a comma to separate the names.

2. Add a new **Style** element. This time, it will be to apply **StoneViewControlTemplate**, as shown in the following listing:

```

<Style TargetType="controls:GamePieceView"

```

```

        Class="Stone">
        <Setter Property="ControlTemplate"
            Value="{StaticResource StoneViewControlTemplate}" />
    </Style>

```

That is all that is required for the stick and stone elements to be added to the game board grid. To add the remaining elements, follow these steps:

1. There are seven rows of sticks: four rows of three and three rows of four. They are nearly identical, but not quite. Locate the **Grid** that defines the gameboard; it will already have the corner dots added. Right after the 16 dot elements, add the following listing for the first row of sticks:

```

<controls:GamePieceView Grid.Row="0" Grid.Column="1" StyleClass="Stick"
    GamePiecePosition="01" GamePieceState="{Binding Game.Sticks[0]}" GamePi
<controls:GamePieceView Grid.Row="0" Grid.Column="3" StyleClass="Stick"
    GamePiecePosition="02" GamePieceState="{Binding Game.Sticks[1]}" GamePi
<controls:GamePieceView Grid.Row="0" Grid.Column="5" StyleClass="Stick"
    GamePiecePosition="03" GamePieceState="{Binding Game.Sticks[2]}" GamePi

```

Each stick in the first row is displayed horizontally. Each stick is given its own position in the **GamePiecePosition** attribute and **GamePieceState** is bound to the **Game.Sticks** object for this stick. The **Sticks** array is zero-based so the indexes for the array are one less than **GamePiecePosition**.

2. Add the code for the second row of sticks using the following listing:

```

<controls:GamePieceView Grid.Row="1" Grid.Column="0" StyleClass="Stick"
    GamePiecePosition="04" GamePieceState="{Binding Game.Sticks[3]}" GamePi
<controls:GamePieceView Grid.Row="1" Grid.Column="2" StyleClass="Stick"
    GamePiecePosition="05" GamePieceState="{Binding Game.Sticks[4]}" GamePi
<controls:GamePieceView Grid.Row="1" Grid.Column="4" StyleClass="Stick"
    GamePiecePosition="06" GamePieceState="{Binding Game.Sticks[5]}" GamePi
<controls:GamePieceView Grid.Row="1" Grid.Column="6" StyleClass="Stick"
    GamePiecePosition="07" GamePieceState="{Binding Game.Sticks[6]}" GamePi

```

These elements are all **Vertical** instead of **Horizontal**; otherwise, they follow the same pattern as the previous step. Keep on going to add the remaining rows.

3. Use the following listing to add the third row of sticks:

```

<controls:GamePieceView Grid.Row="2" Grid.Column="1" StyleClass="Stick"
    GamePiecePosition="08" GamePieceState="{Binding Game.Sticks[7]}" GamePi
<controls:GamePieceView Grid.Row="2" Grid.Column="3" StyleClass="Stick"
    GamePiecePosition="09" GamePieceState="{Binding Game.Sticks[8]}" GamePi
<controls:GamePieceView Grid.Row="2" Grid.Column="5" StyleClass="Stick"
    GamePiecePosition="10" GamePieceState="{Binding Game.Sticks[9]}" GamePi

```

4. Use the following listing to add the fourth row of sticks:

```

<controls:GamePieceView Grid.Row="3" Grid.Column="0" StyleClass="Stick"
    GamePiecePosition="11" GamePieceState="{Binding Game.Sticks[10]}" GameP
<controls:GamePieceView Grid.Row="3" Grid.Column="2" StyleClass="Stick"
    GamePiecePosition="12" GamePieceState="{Binding Game.Sticks[11]}" GameP
<controls:GamePieceView Grid.Row="3" Grid.Column="4" StyleClass="Stick"
    GamePiecePosition="13" GamePieceState="{Binding Game.Sticks[12]}" GameP

```

```
<controls:GamePieceView Grid.Row="3" Grid.Column="6" StyleClass="Stick"
    GamePiecePosition="14" GamePieceState="{Binding Game.Sticks[13]}" GameP
```

5. Use the following listing to add the fifth row of sticks:

```
<controls:GamePieceView Grid.Row="4" Grid.Column="1" StyleClass="Stick"
    GamePiecePosition="15" GamePieceState="{Binding Game.Sticks[14]}" GameP
<controls:GamePieceView Grid.Row="4" Grid.Column="3" StyleClass="Stick"
    GamePiecePosition="16" GamePieceState="{Binding Game.Sticks[15]}" GameP
<controls:GamePieceView Grid.Row="4" Grid.Column="5" StyleClass="Stick"
    GamePiecePosition="17" GamePieceState="{Binding Game.Sticks[16]}" GameP
```

6. Use the following listing to add the sixth row of sticks:

```
<controls:GamePieceView Grid.Row="5" Grid.Column="0" StyleClass="Stick"
    GamePiecePosition="18" GamePieceState="{Binding Game.Sticks[17]}" GameP
<controls:GamePieceView Grid.Row="5" Grid.Column="2" StyleClass="Stick"
    GamePiecePosition="19" GamePieceState="{Binding Game.Sticks[18]}" GameP
<controls:GamePieceView Grid.Row="5" Grid.Column="4" StyleClass="Stick"
    GamePiecePosition="20" GamePieceState="{Binding Game.Sticks[19]}" GameP
<controls:GamePieceView Grid.Row="5" Grid.Column="6" StyleClass="Stick"
    GamePiecePosition="21" GamePieceState="{Binding Game.Sticks[20]}" amePi
```

7. Use the following listing to add the seventh row of sticks:

```
<controls:GamePieceView Grid.Row="6" Grid.Column="1" StyleClass="Stick"
    GamePiecePosition="22" GamePieceState="{Binding Game.Sticks[21]}" GameP
<controls:GamePieceView Grid.Row="6" Grid.Column="3" StyleClass="Stick"
    GamePiecePosition="23" GamePieceState="{Binding Game.Sticks[22]}" GameP
<controls:GamePieceView Grid.Row="6" Grid.Column="5" StyleClass="Stick"
    GamePiecePosition="24" GamePieceState="{Binding Game.Sticks[23]}" GameP
```

8. The sticks are all added, so now we need to add the stones. Use the following listing to add the nine **Stone** elements to the game board **Grid** control following the sticks:

```
<controls:GamePieceView Grid.Row="1" Grid.Column="1" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="1" Grid.Column="3" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="1" Grid.Column="5" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="3" Grid.Column="1" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="3" Grid.Column="3" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="3" Grid.Column="5" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="5" Grid.Column="1" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="5" Grid.Column="3" StyleClass="Stone" GamePieceState="{Bindin
<controls:GamePieceView Grid.Row="5" Grid.Column="5" StyleClass="Stone" GamePieceState="{Bindin
```

This is a wrap on the game app. You can now test out the project in the next section.

Testing the completed project

This project has spanned two chapters, with *Chapter 9, Setting Up a Backend for a Game Using Azure Services*, and this chapter, *Building a Real-Time Game*. Since this is a two-player turn-based game, getting all the components configured correctly can be a challenge. Follow these steps to test out your game locally on Windows:

1. The first step is to get the service running in the background. In Visual Studio, right-click the **SticksAndStones.Functions** project and select **Debug | Start Without Debugging** or press **Ctrl + F5**.

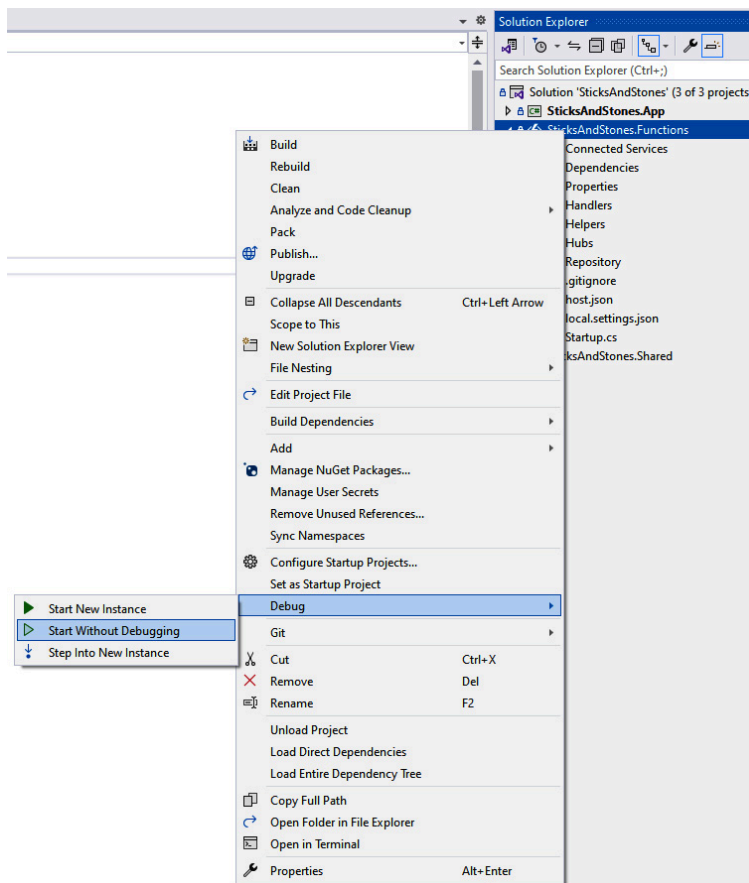


Figure 10.15 – Starting the Azure Functions service

That should launch a terminal window with the Azure Functions service running.

2. Now, two clients are needed to play the game. On Windows, that means the Windows client and the Android client. Start with the Windows client first, and use the same method that was used for Functions. Make sure the Windows target is selected in the **Debug** options:

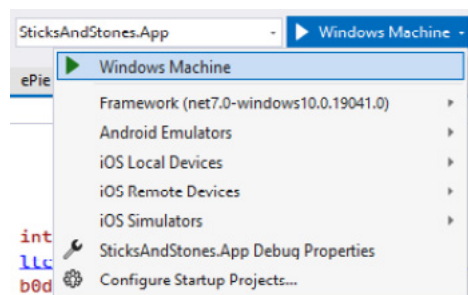


Figure 10.16 – Selecting Windows as the Debug target

3. Right-click the **SticksAndStones.App** project and then select **Debug | Start Without Debugging** or press **Ctrl + F5**. A new window should open with the login page displayed.
4. Now, switch the **Debug** target to Android:

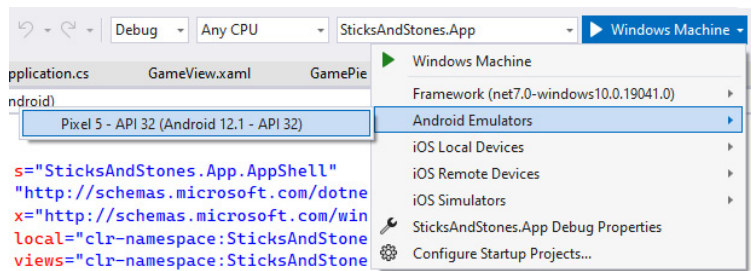


Figure 10.17 – Selecting Android as the Debug target

5. Now, either use *F5* to debug the app in an Android emulator, or *Ctrl + F5* to just run the app.
6. Log in to each app using a different email and gamer tag.

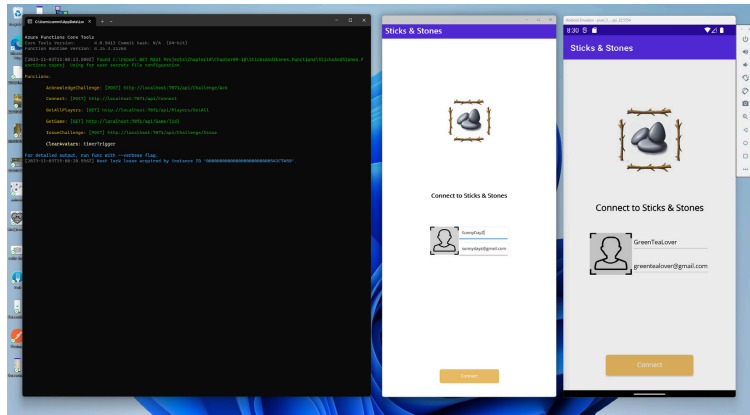


Figure 10.18 – Log in to the game

7. Challenge the other player to a match!

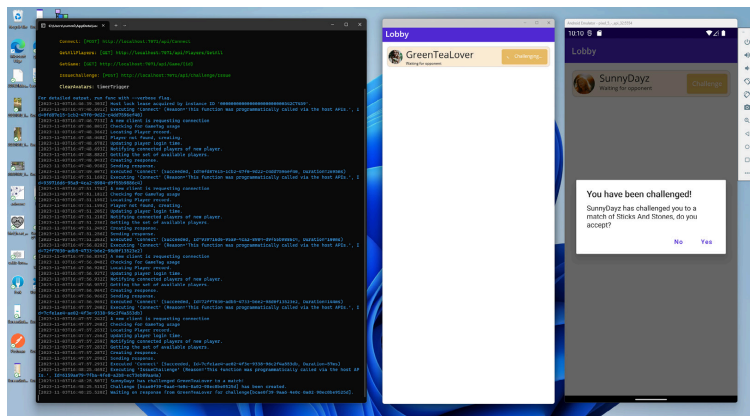


Figure 10.19 – Challenge issued

8. Try to best yourself in a game of *Sticks and Stones*!

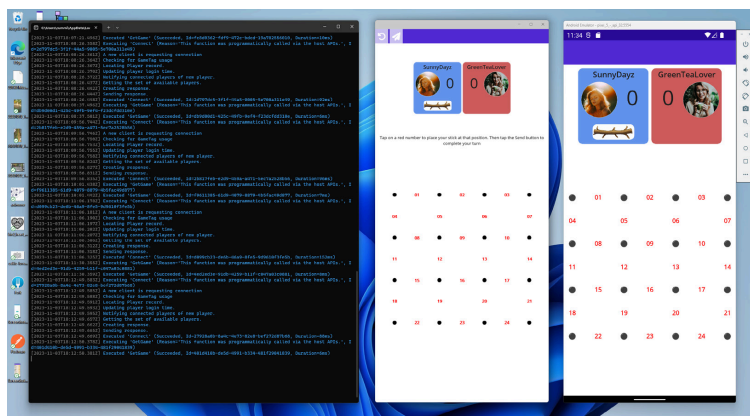


Figure 10.20 – The match has started

ANDROID: CLEARTEXT HTTP TRAFFIC TO 10.0.2.2 NOT PERMITTED

If you are attempting to test the game using the Android client, you will probably hit this error when you try to send a move to the server.

*Fortunately, the resolution is easy. Open the **MainApplication.cs** file in the **Platforms/Android** folder and modify the **Application** attribute on the **MainApplication** class to match the following:*

```
[Application(UsesCleartextTraffic = true)]
```

If you encounter any errors or something just doesn't work the way you expect it to, go back through all the steps and make sure you didn't miss anything. Otherwise, congratulations on making it through this project.

Summary

That's that! Excellent work! There is so much in the chapter that it is hard to keep this summary short. In this chapter, we created a game app that connects to our backend. We created a service that managed the calls to the backend service and handled errors, all asynchronously. We have learned how to respond to messages from SignalR, and how to send and receive messages within the app using the **IMessenger** interface. We created custom controls and used them across multiple pages. We learned how to style an app with XAML styling, how to use control templates, and how to select them using styles. We explored routes and how they work in a multi-page .NET MAUI app. We examined triggers and how we can use them to update the interface without using C# code and converters.

Now, reward yourself and challenge a friend to a match in your new game.

In the next chapter, we will dive into Blazor and .NET MAUI together.